SchoolApp360 — Audit (PARTIAL)

Incomplete: security / performance / devops reports pending

Generated 10 October 2026
Read-only audit — no application source code was modified.
Confidential: contains security findings. Contains no credentials or personal data.

Contents

PARTIAL AUDIT — status and what is missing
- Included in this bundle (7 reports, written by their auditors before the stop)
- NOT produced (missing)
- What this means
- How to finish

14 — Prior audit reconciliation (ADMIN/AUDIT_REPORT.md + ISSUE_CHECKLIST.md vs CURRENT code)
- Counts
- Headline answers
- A. ISSUE_CHECKLIST.md items (51)
- B. Additional concrete claims in AUDIT_REPORT.md not itemised in the checklist
- Notes / residuals found while reconciling (new, owned by A6 or flagged for others)

01 - Repository and Architecture Discovery (Auditor A1)
- Scope
- Method
- Summary of findings
- 1. Tech stack (verified from package.json / lockfile)
- 2. Directory structure and routing
- 3. Prisma schema overview
- 4. Auth and session mechanism (overview; depth belongs to auth auditor)
- 5. Upload / storage (R2)
- 6. Notifications and jobs
- 7. Mobile app existence/structure
- 8. Existing documentation
- 9. Dependency and architecture map
- 10. Feature implementation status
- 11. Environment-variable inventory (names only)
- Positive observations
- Limitations / not assessed

02 — Frontend Engineering and UI/UX Audit (Admin web app)
- Method
- Summary table
- Findings
- Workflow review (structure-level, not executed)
- Frontend security observations
- Positive observations
- Limitations / not assessed

03 — Backend and API audit (auditor A3)
- Scope
- Method and sampling
- Summary table
- Endpoint inventory (180 files, 225 handlers)
- Findings
- Critical flows: handled vs missing
- Positive observations
- Limitations / not assessed

05 - Database and Data Integrity Audit (auditor a5)
- Scope
- Method
- Summary table

Findings
Data-integrity checks run on the local DB (all read-only)
FK / onDelete summary (from pg_constraint, local DB)
Positive observations
Limitations / not assessed

08 — Mobile Application Audit (Expo / React Native, schoolapp360)

Method
Sensitive files (existence and git status only; contents NOT read)
Summary table
Findings
Positive observations
Limitations / not assessed

11 — Observability, operations and privacy audit

Scope
Method
Summary table
Findings
Personal-data inventory (from prisma/schema.prisma)
Positive observations (verified)
Limitations / not assessed

PARTIAL AUDIT — status and what is missing

> **This document set is INCOMPLETE.** The audit was run by six parallel auditors. All six were stopped part-way by an account usage limit (HTTP 429, "weekly limit"), not by a finding or a technical error. What follows is exactly what had been written to disk at that moment. Nothing below has been edited, filled in or extrapolated by the coordinator.

Included in this bundle (7 reports, written by their auditors before the stop)

Report	Auditor area	Findings headings
14 — Prior-audit reconciliation	Re-verification of the earlier 42/100 audit against current code	table of rows
01 — Repository and architecture	Discovery, stack, feature status	5
02 — Frontend and UX	Next.js/React/UX	14
03 — Backend and API	Endpoints, business logic, money flows	21
05 — Database and data integrity	Schema, integrity	13
08 — Mobile application	Expo app	12
11 — Observability and privacy	Logging, audit trail, PII	21

NOT produced (missing)

Deliverable	Reason
04 — Security and tenant isolation	auditor A4 stopped before writing
07 — Security risk register (OWASP, uploads, integrations, `npm audit`)	auditor A4 stopped before writing
06 — Performance and scalability	auditor A5 stopped before writing
13 — Data relationships and module dependencies	auditor A5 stopped before writing
09 — Testing and QA (lint run, test matrix)	auditor A1 stopped before writing
10 — DevOps and production readiness	auditor A1 stopped before writing
12 — Architecture and maintainability	auditor A1 stopped before writing
Consolidated scorecard, de-duplicated risk register, top-20 fixes, remediation roadmap, executive report (`SCHOOLAPP360_COMPREHENSIVE_AUDIT`)	requires all auditors' machine-readable findings; only auditor A2 had written its JSON

What this means

- **Authentication, authorization and tenant isolation have NOT been fully audited in this run.** That is the highest-risk area for a multi-school SaaS, and its report (04) and the OWASP/uploads/webhook register (07) do not exist yet. The partial reports above touch security topics (03 backend, 11 privacy, 14 prior-audit status) but are not a substitute.
- **No overall score, severity totals or production-readiness verdict is given here.** Those need the missing reports; issuing them now would be guesswork.
- Each included report keeps its own "Limitations" section describing what its auditor could not verify.

How to finish

When the usage limit resets, re-run the four unfinished auditors (A4 security first, then A5, A1, and the A3/A6 JSON exports) and then run the consolidation step that produces the executive report and the full PDF.

14 — Prior audit reconciliation (ADMIN/AUDIT_REPORT.md + ISSUE_CHECKLIST.md vs CURRENT code)

Auditor: A6. Read-only. Prior audit score: 42/100, verdict "NOT READY". Every row below was re-checked by re-opening the cited file in the current working tree (code-based; nothing was executed against the app, no POST/mutating request was sent). Line numbers are current-tree lines.

Counts

Total rows: 67 (51 checklist items + 16 additional report claims). FIXED: 24 | PARTIALLY FIXED: 7 | UNVERIFIABLE: 7 | STILL OPEN: 29.

Headline answers

- **Claimed unauthenticated mutating routes (checklist items 2-11, report section 3/4):** all 16 route files / methods named were re-opened. **Every one now begins with a guard** (`requirePermission(...)` or `requireSuperAdmin()`); none is unauthenticated. See rows P2-P11. This verifies guard presence only, not that each permission key is the right one.
- **Hardcoded SuperAdmin password shown on the login page:** `components/SuperAdminLoginForm.jsx` no longer renders any credentials box (re-read lines 1-30 and grep for credential literals: only a code comment about bad credentials). `prisma/seed.js:85-92` now takes the SuperAdmin password from an env var or generates a random one. **FIXED.** Residual: a stale code comment in `lib/auth.js:5-7` still states the old demo credential pair in plain text (documentation leak, not a live credential check); `prisma/seed.js:112-113` still seeds two demo SchoolAdmin accounts with a literal shared password (see P42).
- **`globalThis` tenant singleton:** `lib/currentUser.js:12` and `lib/school.js:17` still exist. Authorization no longer depends on them for the routes the prior audit named (they now use `requirePermission`), but `getCurrentUser()` is still imported as a fallback by attendance and the performance routes, `setCurrentRole` is still called at login (`app/actions/auth.js:92,111,138`) and `lib/auth/schoolContext.js:26` still falls back to the singleton when there is no session (this is what makes the fee cron single-school, see OBS-003). **PARTIALLY FIXED.**
- **Methodology note:** the prior report claimed `Payment` / `StudentFee` use `onDelete: Cascade` from `Student` in one place and `Restrict` in another; the current schema uses Restrict (no `onDelete`) on both (`prisma/schema.prisma:542`, and Payment at 1432) with an explicit friendly error in `lib/students.js:686-690`. The "500 on delete" crash is fixed; the cascade contradiction in the prior text was inaccurate.

A. ISSUE_CHECKLIST.md items (51)

#	Prior finding	Claimed severity	Current status	Evidence (re-read now)
P1	Hardcoded SuperAdmin password on public login page + seed	Critical	FIXED (residual comment)	`components/SuperAdminLoginForm.jsx` has no credential literal; `prisma/seed.js:85-92` env/random password. Residual stale comment `lib/auth.js:5-7`
P2	`POST /api/fees/collect` no auth	Critical	FIXED	`app/api/fees/collect/route.js:6` `requirePermission('fees.collect')`

#	Prior finding	Claimed severity	Current status	Evidence (re-read now)
P3	`POST /api/fees/structures` no auth	Critical	FIXED	`app/api/fees/structures/route.js:10,25` (GET `fees.view`, POST `fees.create`)
P4	`/api/fees/students/[id]/discount` no auth	Critical	FIXED	`.../discount/route.js:6,20` `fees.update` on POST and DELETE
P5	`PUT /api/school/profile` no auth	Critical	FIXED	`app/api/school/profile/route.js:6` `settings.update`
P6	branding/contact/preferences/attendance-settings no auth	Critical	FIXED	each `app/api/school/<x>/route.js:6` `settings.update` (all four re-opened)
P7	`/api/sessions` + `[id]` activate/archive no auth	Critical	FIXED	`app/api/sessions/route.js:6`, `sessions/[id]/route.js:6`, `activate/route.js:6`, `archive/route.js:6`
P8	`admins/[id]/reset-password` no auth	Critical	FIXED	`app/api/admins/[id]/reset-password/route.js:6` `requireSuperAdmin()`
P9	`admins/[id]/permissions` no auth	Critical	FIXED	`.../permissions/route.js:6` `requireSuperAdmin()`
P10	`admins/[id]/status` no auth	Critical	FIXED	`.../status/route.js:6` `requireSuperAdmin()`
P11	`schools/[id]/manage` no auth	Critical	FIXED	`app/api/schools/[id]/manage/route.js:8` `requireSuperAdmin()`
P12	`globalThis.__CURRENT_USER__` drives authorization	Critical	PARTIALLY FIXED	still defined `lib/currentUser.js:12`; routes gate on `requirePermission` first (e.g. `app/api/attendance/route.js:17`, `performance/yearly/route.js:9`) but fall back to `getCurrentUser()` (`attendance/route.js:31`, `yearly/route.js:19`); `setCurrentRole` still called `app/actions/auth.js:92,111,138`
P13	`getActiveSchoolId()` default in students/teachers	Critical	PARTIALLY FIXED	no default left in `lib/students.js` / `lib/teachers.js` (only a comment `teachers.js:435`); but `lib/auth/schoolContext.js:26` still falls back to the singleton when no session (affects cron, OBS-003)
P14	Student/StudentFee/Payment delete crash, no soft-delete	Critical	FIXED	`StudentStatus` has TransferredOut/Graduated `schema.prisma:343-348`; FK Restrict + friendly message `lib/students.js:673-692`. Not executed. Erasure side effects: PRV-004
P15	`JWT_SECRET` demo value in prod	Critical	UNVERIFIABLE	production env not accessible. Locally `.env` is git-ignored (`git check-ignore` -> ignored, not in `git ls-files`) but still carries a comment self-labelling the secret "demo-only". Must be rotated/confirmed in Vercel
P16	Fee summary null-actor bypass	Critical	FIXED (sibling routes still have the pattern)	`app/api/fees/student/[studentId]/summary/route.js:13-19` requires actor. Same `actor?.role === 'Parent'` null-skip remains in `app/api/payments/razorpay/create-order/route.js:13` and `verify/route.js:12` (see OBS-007)

#	Prior finding	Claimed severity	Current status	Evidence (re-read now)
P17	No rate limiting on logins	High	PARTIALLY FIXED	`lib/auth/loginRateLimit.js` (in-memory Map, 8/acct, 20/IP per 15 min) used by `app/actions/auth.js`, parent/teacher/staff login routes. Per-instance, resets on cold start (its own header says so); no alerting
P18	`Access-Control-Allow-Origin: *` on `/api/**`	High	STILL OPEN	`middleware.js:61` wildcard; comment `:44-48` argues Bearer-only. Practical risk is low (browsers do not send cookies with a wildcard origin) but the config is unchanged
P19	`ADMIN_PERMISSIONS` decorative	High	PARTIALLY FIXED	enforcement is now RBAC (`lib/rbac.js` `requirePermission`, `prisma/rbacPermissions.js`); `ADMIN_PERMISSIONS` in `lib/adminConstants.js:7` is only imported by `lib/admins.js:4` (defaults), still not enforced. UI not re-checked
P20	`getSchoolStudents` unbounded	High	FIXED	`app/dashboard/students/page.jsx:2,70` uses `getStudentsPage`; `app/api/students/paged/route.js:27`
P21	`getSchoolTeachers` unbounded	High	FIXED	`app/dashboard/teachers/page.jsx:4,17`; `lib/teachers.js:185-218` skip/take
P22	Fees 8+ unbounded `findMany`	High	PARTIALLY FIXED	paging added `lib/fees.js:789,1304-1305`; original sites not individually re-verified; `findMany` total across `lib/*.js` now 146
P23	Attendance query no date bound	High	FIXED (cited site)	`lib/attendance.js:178-181` `date: {gte, lte}`
P24	Students page fetch-all-then-slice	High	FIXED	`app/dashboard/students/page.jsx:70` server-side page
P25	Teachers page same	High	FIXED	`app/dashboard/teachers/page.jsx:17`
P26	Raw `<img>` everywhere	High	STILL OPEN	19 `<img` occurrences remain in `components/` + `app/`; `next/image` appears in only 3 files (login screens)
P27	Single migration, no `migrate deploy`	High	STILL OPEN	`prisma/migrations/` has one folder + lock file; `package.json:7` build = `prisma generate && next build`
P28	No refund workflow	High	STILL OPEN	only comments mention it (`lib/fees.js:850,870`); permission key `fees.refund` exists (`prisma/rbacPermissions.js`) but no refund function/route (`grep -ri refund app/api` -> none)
P29	Late fee never charged	High	FIXED	`lib/fees.js:119-142` `lateFeeAmountFor` folded into payable amount
P30	No server-side R2 size enforcement	Medium	STILL OPEN	`lib/storage.js:33-40` PutObject with no ContentLength condition; size only compared to client-reported `fileSize` (`app/api/students/document-upload-url/route.js:22`)
P31	zod schemas unused by API	Medium	STILL OPEN	`grep -rln "lib/schemas" app` -> 0 files under `app/api`; schemas used only by client forms. `lib/schemas.js` is 659 lines
P32	`Notice` missing `@@index([schoolId])`	Medium	STILL OPEN	`schema.prisma:916-~972` Notice has only `@@map`

#	Prior finding	Claimed severity	Current status	Evidence (re-read now)
P33	`CalendarEvent` missing index	Medium	STILL OPEN	`schema.prisma:892-914` only `@@map`
P34	No `error.jsx`	Medium	STILL OPEN	`find app -name error.jsx -o -name global-error.jsx` -> none; only `app/not-found.jsx`
P35	`loading.jsx` missing on main routes	Medium	FIXED	40 `loading.jsx` files now incl. students, teachers, fees, exams, attendance, dashboard home
P36	`aria-label` near-zero	Medium	STILL OPEN	2 `aria-label` lines in all `components/`
P37	No focus trap in Modal	Medium	STILL OPEN	`components/Modal.jsx` has no focus management (grep for focus/Tab -> no matches)
P38	No admissions workflow	Medium	STILL OPEN	no admissions/enquiry route group in `app/api` listing
P39	Absent forces 0 before retake merge	Medium	STILL OPEN (business rule; not re-judged)	`lib/examResults.js:187-194` absent -> 0 and `allSubjectsPass=false`; retake policy applied later `:259-281`
P40	Attendance unlock no deadline re-check	Medium	STILL OPEN	`lib/attendance.js:72-79` only toggles `manuallyUnlocked`; `app/api/attendance/[id]/lock/route.js:14-22` only permission + existence check
P41	`GET /api/exams/[id]` not school-scoped	Medium	FIXED	`lib/exams.js:156-160` `findFirst({id, schoolId})`; route `app/api/exams/[id]/route.js:8-13`
P42	Seed demo credentials	Low	STILL OPEN	`prisma/seed.js:112-113` two demo admin accounts with literal shared password; whether ever seeded in prod is UNVERIFIABLE
P43	Split `lib/api.js`	Low	STILL OPEN	now 2,181 lines (was 1,514)
P44	Split `lib/fees.js`	Low	STILL OPEN	now 1,728 lines (was 1,118)
P45	Move `studentSeedData.js` out of lib	Low	STILL OPEN	`lib/studentSeedData.js` 816 lines still in `lib/`
P46	Split `lib/schemas.js`	Low	STILL OPEN	659 lines
P47	Shared stats helper students/teachers	Low	UNVERIFIABLE	not re-read (duplication claim; low value)
P48	Mobile responsive coverage on two components	Low	STILL OPEN (by prior's own metric)	breakpoint-class count: `TeachersExplorer.jsx` 0, `StudentFeeDetailPanel.jsx` 1. Not visually tested

#	Prior finding	Claimed severity	Current status	Evidence (re-read now)
P49	Push retry/dead-letter	Low	STILL OPEN	`lib/expoPush.js:41-61` prunes `DeviceNotRegistered` tokens and logs other errors; no retry, no delivery record (OBS-006)
P50	No error tracking	Low	STILL OPEN (web)	no Sentry/APM in web `package.json`; mobile app does have `@sentry/react-native` (`schoolapp360/src/core/sentry.js`). OBS-001
P51	Backup/restore strategy undocumented	Low	STILL OPEN / UNVERIFIABLE	only `docs/API-Spec.md` exists; no backup/DR/incident doc in repo. Provider-side backups not assessable. OBS-010

B. Additional concrete claims in AUDIT_REPORT.md not itemised in the checklist

#	Prior finding	Claimed severity	Current status	Evidence
R1	No shared auth middleware for `/api/**`; each route self-guards	High (design)	STILL OPEN	`middleware.js:55-66` only applies CORS for `/api/`; routes self-guard
R2	Attendance routes (4) have no real-session fallback, run off global	Critical	FIXED	`attendance/route.js:17` and `attendance/[id]/lock/route.js:14` use `requirePermission`; global only a post-auth fallback (see P12)
R3	Role check in lock route used global toggle	High	FIXED	`lock/route.js:14`
R4	CORS `*` with cookie routes	High	STILL OPEN	see P18
R5	Inconsistent 404 vs 403	Low	UNVERIFIABLE	not systematically re-sampled
R6	Student/Teacher/Accountant/Staff roles don't exist	Medium	FIXED (changed)	`lib/rbacConstants.js:6` roles: SuperAdmin, Principal, Admin, Teacher, Accountant, Parent
R7	One Active session per school only app-level	Medium	STILL OPEN	`lib/academicSessions.js:132-133` two sequential writes, not a transaction; no partial unique index
R8	`Teacher.assignments` Json scanned in JS	Medium	PARTIALLY FIXED	class filter now SQL jsonb `@>` (`lib/teachers.js:203-207`); authorization scope helpers still read Json in JS
R9	"Step into school" has no audit log	Medium	STILL OPEN	`app/api/schools/[id]/manage/route.js:8-25` guarded but writes no log (OBS-008)
R10	No security headers	Medium	STILL OPEN	`next.config.mjs` is empty; no X-Frame/CSP/HSTS in `middleware.js`
R11	`vercel.json` `syd1` pin silently ignored on Hobby	Medium	UNVERIFIABLE	Vercel plan not visible from code; `vercel.json:2` still pins `syd1`
R12	SMTP is Gmail/dev-tier	Medium	UNVERIFIABLE	env values intentionally not read; code supports generic SMTP (`lib/email.js`) and dev-log fallback (OBS-011)
R13	Dashboard unbounded Promise.all queries (`lib/dashboard.js:304-322`, `lib/exams.js:311-321`)	Medium	UNVERIFIABLE	not re-measured (performance auditor's area)

#	Prior finding	Claimed severity	Current status	Evidence
R14	`lib/classes.js` mixes roster queries; `lib/api.js` generically named	Low	UNVERIFIABLE	not re-read
R15	Cross-tenant race via manage route + role toggle	Critical	PARTIALLY FIXED	manage route now SuperAdmin-guarded (P11), but `setActiveSchoolContext` / `setCurrentRole` still mutate process-wide state (`manage/route.js:21,24`); a SuperAdmin "step in" still changes the fallback school for every no-session code path on that instance
R16	Multi-school teacher/parent accounts unsupported	Info	STILL OPEN (design)	`Teacher.schoolId` , `ParentAccount.schoolId` singular (`schema.prisma:271-286`)

Notes / residuals found while reconciling (new, owned by A6 or flagged for others)

- `app/api/payments/razorpay/create-order/route.js:13` and `verify/route.js:12` : ownership check is skipped when no actor is signed in (same pattern the prior audit flagged for the fee summary). Code-based; see OBS-007.
- `lib/auth.js:5-7` : comment retains the old demo credentials in clear text.
- `dev.log` (46 KB) is tracked in git (`git ls-files`), see OBS-012.
- Prior audit's "decorative permission" and "fee routes unguarded" findings were reflected by a large RBAC refactor (task comments in `prisma/rbacPermissions.js`). The guards exist; correctness of each permission-to-route mapping belongs to the authorization auditor.

01 - Repository and Architecture Discovery (Auditor A1)

Scope

Phase-zero discovery of `ADMIN` (`C:\Users\Tashaf\Desktop\SchoolMS\adminpanel`) and a structural look at `MOBILE` (`..\schoolapp360`). Covers tech stack, structure, auth/session overview, storage, notification infrastructure, docs, a feature implementation-status table and an environment-variable inventory. Read-only; code-based unless stated.

Method

- Read `package.json`, `package-lock.json` (xlsx entry), `vercel.json`, `next.config.mjs`, `middleware.js`, `lib/db.js`, `lib/auth/*`, `lib/rbac.js`, `lib/iam.js`, `lib/storage.js`, `lib/email.js`, `lib/pusher.js`, `lib/expoPush.js`, cron routes, `prisma/schema.prisma` (headers + model/enum names), `prisma/migrations`.
- Globs/greps across `app`, `lib`, `components` for route handlers, pages, loading/error files, `process.env.*`, `globalThis`, and import references.
- Next.js version is 16.3.2. Consulted `node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/proxy.md` (line 11: `middleware` is deprecated and renamed `proxy`; changelog line 774: Proxy defaults to Node.js runtime in v16.0.0).
- Feature status below is from tracing UI page -> API route -> lib -> Prisma model by reading code. Nothing was executed against the app; no runtime verification except where noted.
- Counts verified with `find` / `wc`: 180 `route.js`, 70 `page.jsx`, 74 `lib/*.js` (incl. subdirs, 5 more in `lib/auth`, `lib/hooks`, `lib/performance`), 283 component `.jsx`, 40 `loading.jsx`, 1 `not-found.jsx`, 0 `error.jsx` / `global-error.jsx`, 52 Prisma models, 31 enums.

Summary of findings

ID	Sev	Conf	Title	Component
ARC-001	P1	Medium	Student documents (birth cert, Aadhaar, SLC) uploaded to a public-read R2 prefix with predictable keys	Storage / privacy
ARC-002	P2	High	No Razorpay webhook: payment recording depends on the browser returning to the verify endpoint	Fees / payments
ARC-003	P3	High	`middleware.js` uses the file convention deprecated in Next 16 (renamed `proxy`)	Framework
ARC-004	P3	High	No `error.jsx` / `global-error.jsx` anywhere in `app/`	Frontend resilience
ARC-005	P3	Medium	`GeneratedReportJob` model (and 3 seed-data libs, 8 components) are dead	Schema / repo

(See `12-architecture-and-maintainability-audit.md` for MNT-xxx and `10-devops-and-production-readiness.md` for OPS-xxx.)

---

1. Tech stack (verified from `package.json` / lockfile)

Layer	Choice	Version
Framework	Next.js App Router (Turbopack dev)	16.3.2
UI	React / react-dom	19.2.8
Language	Plain JavaScript (`.js` / `.jsx`, `jsconfig.json`), no TypeScript	-

Layer	Choice	Version
Styling	Tailwind CSS 4 + `@tailwindcss/postcss`	^4
ORM / DB	Prisma Client + CLI, PostgreSQL	6.19.3 (pinned)
Auth	`jose` HS256 JWT in cookie or Bearer header, `bcryptjs`	jose ^6.2.10, bcryptjs ^3.0.3
Forms / validation	react-hook-form + @hookform/resolvers + zod	^7.86 / ^5.9 / ^4.4
Payments	`razorpay` SDK (per-school keys)	^2.9.8
Realtime	`pusher` (server) + `pusher-js` (browser), used only for Support tickets	^5.3.4 / ^8.6.0
Mobile push	Expo Push HTTP API (raw `fetch`, no SDK)	-
Email	`nodemailer` (SMTP, console fallback)	^10.0.1
Storage	Cloudflare R2 via `@aws-sdk/client-s3` + presigner	^3.1128
Spreadsheets	`xlsx` (SheetJS) from CDN tarball	0.20.3
Charts / icons	chart.js + react-chartjs-2; react-icons (223 importing files)	-
Lint	eslint 9 + `eslint-config-next/core-web-vitals`	16.3.2
Hosting	Vercel, region `syd1`, 2 crons (`vercel.json`)	-

Mobile (`schoolapp360`): Expo ~57, React Native 0.86.3, React 19.2.3, Redux Toolkit, React Navigation 7, NativeWind, `@sentry/react-native` ~7.11, `react-native-razorpay`, `expo-notifications`, jest ~29.7 + `@testing-library/react-native`. 104 files under `src/`, own git repo (55 tracked files, 67 working-tree changes).

2. Directory structure and routing

```
adminpanel/
  app/
    layout.jsx, page.jsx (login redirect), not-found.jsx, globals.css
    login/ forgot-password/ set-password/ setup/        public auth pages
    dashboard/   (SchoolAdmin/Teacher/Principal/Accountant) 52 page.jsx
    parent/      (Parent portal) with (home) and exams/(list) route groups
    super-admin/ (SuperAdmin: schools, admins, users, support)
    actions/auth.js   the ONLY Server Actions file (login/logout actions)
    api/**/route.js   180 route handlers across 34 top-level resources
  components/  283 .jsx, grouped by domain (students, fees, exams, ...)
  lib/         74 modules: domain data layer + auth/ hooks/ performance/
  prisma/      schema.prisma (1,775 lines), seed.js, migrate-rbac.js, rbacPermissions.js, migrations/
(1 file)
  middleware.js  root-level (deprecated convention, see ARC-003)
  store/ scripts/  both empty
  docs/API-Spec.md, README.md (create-next-app boilerplate), design.md, SKILL.md (1,228 lines)
```

- **Route groups:** only `(home)` and `exams/(list)` under `/parent`. Separation is by top-level segment (`dashboard`, `parent`, `super-admin`), not by group.
- **Layouts:** `app/layout.jsx`, `app/dashboard/layout.jsx` (loads both `getCurrentUser()` and `getCurrentUserInfo()`), `app/parent/layout.jsx`, `app/super-admin/layout`-equivalent. Feature gating via `lib/featureFlagGuard.js` (flags: fees, exams, homework, leave).
- **Loading boundaries:** 40 `loading.jsx` files. **Error boundaries:** none (ARC-004).
- **Middleware (`middleware.js`):** matcher covers `/api/:path*`, `/dashboard`, `/super-admin`, `/parent`, `/`, `/login`, `/forgot-password`. For `/api/*` it only adds CORS headers (`Access-Control-Allow-Origin: *`, lines 56-67) and answers OPTIONS; it performs NO authentication for APIs. Page-level gate: verifies JWT and redirects by role. All API authorization is therefore per-handler.
- **Route handlers vs server actions:** 180 route handlers carry all data mutations; server actions are used only for the five login/logout flows in `app/actions/auth.js`. Client components call `lib/api.js` (2,181 lines, 190

exported fetch wrappers) which hits the route handlers.

> **Route-handler authorization coverage (code-based grep, 180 handlers)**

- 101 use `requirePermission`, 13 `requireSuperAdmin`, 60 `getCurrentUserInfo`, 2 cron-secret handlers.
- Only 3 handlers contain no recognizable guard keyword: `auth/parent/login` (expected public), and `students/bulk-import/template` + `blank-template` (static Excel templates; both call `resolveSchoolId()` per grep of related code but need a closer review by the auth auditor). This is a large improvement over the previous audit's "~10 unauthenticated mutating routes" claim, which I did NOT reproduce; a keyword match is not proof of correct per-route checks, which belongs to the auth/authorization auditors.

3. Prisma schema overview

- `provider = "postgresql"`, `url = env("DATABASE_URL")` only (no `directUrl`), `binaryTargets = ["native","rhel-openssl-3.0.x"]` (`prisma/schema.prisma:14-21`).
- 52 models, 31 enums. Domains: identity/RBAC (`SuperAdmin`, `SchoolAdmin`, `User`, `Role`, `Permission`, `RolePermission`, `ParentAccount`, `ParentStudentLink`), tenant (`School`), people (`Student`, `Teacher`, `StudentEnrollment`, `StudentPromotionLog`), academics (`Class`, `Section`, `Subject`, `TimeTable`, `AcademicSession`, `CalendarEvent`), attendance (`Attendance`, `TeacherAttendance`, `TeacherLeave`), exams (`Exam`, `ExamSchedule`, `ExamMark`, `ExamResult`, `ExamAuditLog`, `GradeScale`, `ExamOptionalEnrollment`), fees (`FeeStructure*`, `StudentFee*`, `Payment`), performance reports (`SubjectTest*`, `MonthlyReport*`, `YearlyReport`, `ReportLock`, `GeneratedReportJob`), comms (`Notice`, `Homework`, `PushToken`, `ParentNotification`, `TeacherNotification`, `AdminNotification`), support (`SupportTicket`, `SupportTicketReply`).
- 49 `@@index` / `@@unique` declarations. `Payment.razorpayOrderId` and `razorpayPaymentId` are `@unique` (`schema.prisma:1438-1439`), which gives DB-level duplicate-payment protection.
- **Migrations:** single migration `20260830172048_init` of 38 lines creating only `SuperAdmin` and `SchoolAdmin`. The other 50 models are not covered by any tracked migration (OPS-003).

4. Auth and session mechanism (overview; depth belongs to auth auditor)

- JWT HS256, 7-day expiry, signed with `JWT_SECRET` (`lib/auth/jwt.js:1-27`). Payload is trusted verbatim; not re-derived from DB on every request (comment lines 12-14).
- Cookie `schoolapp360_session`, `httpOnly`, `secure` in production, `sameSite: 'lax'` (`lib/auth/session.js:10-20`). `getSession()` falls back to `Authorization: Bearer` for mobile (`session.js:30-39`).
- Logins: Server Actions for SuperAdmin/SchoolAdmin/Teacher/Parent (`app/actions/auth.js`), plus REST `api/auth/{parent,teacher,staff}/login` for mobile. In-memory rate limiter (`lib/auth/loginRateLimit.js`, see OPS-005).
- RBAC: `lib/rbac.js` exposes `requireAuth/requireRole/requirePermission/requireAnyPermission`; permission keys seeded from `prisma/rbacPermissions.js`; roles `Principal/Admin/Accountant/Teacher/SuperAdmin/Parent`.
- **Dual identity model:** legacy `role` (SuperAdmin/SchoolAdmin/Teacher/Parent) plus newer `roleKey`, and two "current user" resolvers (`lib/currentUser.js` demo toggle and `lib/iam.js`). Covered in MNT-001/MNT-002.

5. Upload / storage (R2)

- `lib/storage.js`: new `S3Client` per call; `getPresignedUploadUrl` (PUT, 300 s expiry, `ContentType` signed but no `ContentLength`/size condition), `deleteObject` (swallows errors), `listObjectKeysWithPrefix`, `buildPublicUrl` (`${R2_PUBLIC_URL}/${key}`).
- 10 presign routes (`admin|parent|teacher/profile-photo-upload-url`, `school/logo-upload-url`, `students/{photo,document}-upload-url`, `teachers/{photo,document}-upload-url`, `notices/upload-url`, `support/upload-url`). The type and size checks use client-declared `fileType` / `fileSize` (`students/document-upload-url/route.js:19-25`), so a client can lie about the size and upload anything of the declared content-type.

- No signed GET URLs: all reads go straight to a public bucket URL (`storage.js:20-26` comment: bucket "already set to allow public reads").

> **ARC-001 - Student documents stored in a public-read bucket with guessable keys**

- **Category:** Security / Privacy
- **Severity:** P1 **Confidence:** Medium **Status:** Suspected (code-based; bucket ACL/config not verified)
- **Affected component:** Student document uploads (father/mother photo, birth certificate, SLC, Aadhaar), notices attachments, support attachments
- **Evidence:** `lib/storage.js:20-26` (public URL, comment requires public-read bucket); `app/api/students/document-upload-url/route.js:28` key = `students/<schoolId>/documents/<Date.now()>-<safeName>`; `app/api/support/upload-url/route.js:33` same pattern; `route.js:19-25` size/type trusted from the client.
- **Impact:** identity documents of minors/parents are readable by anyone who obtains or guesses the URL (millisecond timestamp plus a usually predictable filename such as `aadhaar.pdf`); the URL is persisted and returned to every client that renders the student. No access revocation or audit is possible; deleting a student does not necessarily delete the object. This is a privacy/regulatory exposure (India's DPDP Act for children's data).
- **Reproduction / verification:** code-based; not executed. To confirm: check R2 bucket public-access setting and whether `R2_PUBLIC_URL` is an `r2.dev` /custom domain with public read.
- **Root cause:** public-bucket design chosen for simplicity of display; keys carry no unguessable component.
- **Recommended fix:** minimum: add `crypto.randomUUID()` to every key. Better: keep sensitive prefixes (`students/*/documents/`, Aadhaar) in a private bucket and serve through an authenticated route that returns a short-lived presigned GET (`@aws-sdk/s3-request-presigner` is already installed). Also pass `ContentLength` via a presigned POST policy or re-check the object size with `HeadObject` on confirm.
- **Validation:** unauthenticated GET of a stored document URL returns 403; authenticated route returns 200 with expiring URL.
- **Effort:** M **Dependencies:** data migration of existing keys; MNT-003.

6. Notifications and jobs

Mechanism	Implementation	Retry / durability
Expo push	`lib/expoPush.js` batches of 100, `Promise.all`; deletes token on `DeviceNotRegistered`; logs other errors	Fire-and-forget, no retry, no receipts check
Pusher	`lib/pusher.js` lazy singleton; only Support tickets channels (`support-school-*`, `support-superadmin`, `support-ticket-*`)	Best-effort
Email	`lib/email.js` nodemailer; no `SMTP_HOST` => message only logged to console (comment says production provider is a deferred decision)	No retry, no queue
In-app notifications	DB tables `ParentNotification`, `TeacherNotification`, `AdminNotification` written by `lib/*Notifications.js`	Durable (DB)
Crons	`vercel.json`: `/api/cron/generate-fees` 03:00 UTC, `/api/cron/cleanup-support-tickets` 04:00 UTC; both GET with optional bearer check	See OPS-001

No queue/outbox exists. A serverless function that returns before the `fetch` to Expo/Pusher resolves may drop the send (no `waitUntil` / `after` used: grep found none).

7. Mobile app existence/structure

`..\schoolapp360`: Expo app, `src/{api,components,core,features,hooks,theme,utils}`, `App.js`, `android/` native project, Sentry wired in `src/core/sentry.js` (enabled only when `EXPO_PUBLIC_ENABLE_CRASH_REPORTING==='true'` and a DSN is set), `jest.config.js` + `jest.setup.js`, 0 test files, many root-level docs (`ARCHITECTURE.md`,

`SETUP.md`, `TEST_FIX.md`, `PROJECT_SETUP_SUMMARY.md`, `README_COMPLETE.md`, ...). The jest config is currently broken (see 09 report TST-002).

8. Existing documentation

Doc	State
`README.md` (36 lines)	create-next-app boilerplate; no setup, env, deploy or architecture info
`docs/API-Spec.md` (1,175 lines)	Detailed endpoint spec; line 1147 says one ledger route does not reject unauthenticated callers with 401 - must be re-verified before use as truth
`SKILL.md` (1,228 lines)	Agent-oriented project description; the only real architecture narrative; line 818+ describes login behaviour that predates current code
`design.md` (115 lines)	design tokens/UI guidance
`AUDIT_REPORT.md` / `ISSUE_CHECKLIST.md` (+PDF)	Prior audit (score 42/100). Not copied; findings referenced by other auditors. Spot check: `lib/auth.js` no longer holds hardcoded credentials; `prisma/seed.js:88` now takes `SUPER_ADMIN_SEED_PASSWORD` or a random value. Residual: a plaintext password remains in a code comment (OPS-015 in report 10).
`EXAM_MODULE_UI_PROMPT.md`	prompt notes
Architecture / runbook / ER diagram	absent (MNT-007)

9. Dependency and architecture map

```
Browser (Client Components, react-hook-form + zod)
   |  lib/api.js (fetch wrappers, 190 fns)            Mobile app (Bearer JWT)
   v                                                   v
middleware.js  --(pages: JWT role redirect; /api: CORS only)--
   |
app/api/**/route.js (180)  -- requirePermission / getCurrentUserInfo / requireSuperAdmin
   |
lib/<domain>.js  (students, fees, exams, attendance, notices, ...)   lib/rbac.js, lib/iam.js,
lib/auth/*
   |                                  |-- lib/razorpay.js (per-school keys, AES-GCM at rest:
lib/auth/encryption.js)
   v                                  |-- lib/storage.js -> Cloudflare R2
lib/db.js (globalThis PrismaClient) --> PostgreSQL (DATABASE_URL)
                                      |-- lib/pusher.js, lib/expoPush.js, lib/email.js
Server Component pages (app/dashboard/**, app/parent/**) call lib/* directly (skipping route handlers)
Crons (vercel.json) -> /api/cron/* -> lib/fees.js, lib/support.js
```

Layering is mostly pages -> lib -> prisma. `lib/*Constants.js` files are intentionally client-safe; several lib files must not import server-only modules (documented in comments of `lib/school.js` and `lib/featureFlags.js`), an import-hygiene constraint enforced only by comments.

10. Feature implementation status

Legend: Fully / Partially / Mocked / Duplicated / Unused / Missing / Cannot verify. Code-read, not executed.

Feature	Status	Trace evidence	Notes
Students (CRUD, list, bulk import, print details)	Fully	`app/dashboard/students/*`, `app/api/students/{route,paged,bulk-import,print-details}`, `lib/students.js` (931 lines), `Student` model	Server-side pagination via `students/paged`.
Enrollment and promotion	Partially	`app/dashboard/students/promote`, `api/students/promote`, `lib/students.js:738` `promoteStudents`, models `StudentEnrollment`, `StudentPromotionLog`, `AcademicSession`	Implemented; no automated test, rollback/undo of a promotion not found (grep).
Teachers / staff	Fully	`app/dashboard/teachers/*`, `api/teachers/*`, `lib/teachers.js`, teacher login	
Classes / sections / subjects / sessions	Fully	`app/dashboard/{classes,academics,sessions}`, `lib/classes.js`, `lib/subjects.js`, `lib/academicSessions.js`	Three seed-data libs for these are unreferenced (dead).
Student attendance	Fully	`app/dashboard/attendance/{daily,reports}`, `api/attendance/*`, `lib/attendance.js`, lock/unlock `api/attendance/[id]/lock`	Teacher scope check in route.
Staff attendance	Fully	`dashboard/attendance/staff`, `api/staff-attendance`, `lib/teacherAttendance.js`, `TeacherAttendance` model, `TeacherCheckInCard`	Location helper `lib/location.js`.
Exams / marks / results	Fully (workflow)	`dashboard/{exams,marks-entry}`, `api/exams/[id]/{verification,results,status,audit-log,duplicate,schedules}`, `lib/exams*.js`, `lib/examMarks.js`, `ExamAuditLog`	Marks -> verify -> publish present; transitions untested.
Report cards	Partially	`dashboard/print-marksheet`, `api/print-marksheet`, `lib/printMarksheet.js`; monthly/yearly performance reports under `dashboard/performance/*`	`GeneratedReportJob` model has no code references (unused; background generation never built).
Fees / payments / discounts / late fees	Fully, with gap	`dashboard/fees/*`, `api/fees/*`, `api/payments/*`, `lib/fees.js` (1,728 lines), `lateFee*` fields, `DiscountModal`	Online payment relies on client-return verify; no webhook (ARC-002). Cron auto-generates term fees.
Timetable	Fully	`dashboard/academics/timetable`, `api/timetable`, `lib/timetable.js`, `TimeTableClient.jsx` (1,023 lines)	
Homework	Fully	`dashboard/homework`, `api/homework/{route,page}`, `lib/homework.js`, parent view	Feature flag `homework`.
Notices / notifications	Partially	`dashboard/notices`, `api/notices/{page,paged,upload-url}`, `lib/notices.js`, `lib/expoPush.js`, notification tables	`notices/page` and `notices/paged` are two near-duplicate list endpoints (the `page` route comment says `paged` is kept for the mobile app). No retries.
Support tickets	Fully	`super-admin/support`, `api/support/tickets`, `lib/support.js`, Pusher, cleanup cron	Newest module; untested.
Leaves	Partially	`dashboard/leave`, `api/leaves`, `lib/teacherLeaves.js`, `TeacherLeave`	Teacher leave only; no student/parent leave found.
Calendar	Fully	`dashboard/academics/calendar`, `api/calendar-events`, `api/{parent,teacher}/calendar-events`	

Feature	Status	Trace evidence	Notes
Reports / exports	Partially	Attendance report table, performance report import templates (`xlsx`), student bulk-import templates	Import templates and tables exist; no generic export/CSV download service found; Cannot verify PDF export (`print-marksheet` is browser print).
RBAC / roles	Partially	`lib/rbac.js` , `dashboard/settings/roles` , `api/{roles,permissions,users,iam}` ; 101/180 handlers use `requirePermission`	Dual legacy `role` + `roleKey` ; `middleware.js:22-37` itself says the roleKey path is "not yet reachable".
Multi-school / tenant model	Partially	`School` model, `schoolId` on tenant tables, `resolveSchoolId()` from session	Process-global `ACTIVE_SCHOOL` fallback (MNT-001); cross-tenant tests absent.
Parent portal	Fully	`app/parent/*` (home, attendance, calendar, exams, fees w/ Razorpay card, homework, notices, profile), `api/parent/*` , `ParentStudentLink`	
SuperAdmin	Fully	`app/super-admin/{schools,admins,users,support}` , `api/schools/*` (per-school Razorpay config, feature flags, manage/step-in)	"Manage this school" step-in mutates process-global state (MNT-001).
Online fee webhook	Missing	no route under `api/payments/razorpay` besides `create-order` and `verify` ; comment at `lib/fees.js:1257` says "a webhook, once built"	ARC-002
Email delivery	Partially	`lib/email.js` falls back to console log without SMTP_HOST	Production provider unconfigured/unknown.

ARC-002 - No Razorpay webhook; successful payments depend on the browser/app calling verify

- **Category:** Backend / Financial integrity
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** Online fee payment
- **Evidence:** `app/api/payments/razorpay/` contains only `create-order/route.js` and `verify/route.js` ; `lib/fees.js:1194-1265` `verifyRazorpayPayment` is idempotent (SUCCESS short-circuit at ~1214, unique `razorpayPaymentId` P2002 handling at ~1257-1262) but only runs when the client calls it; the comment at 1257 anticipates a webhook "once built".
- **Impact:** if a parent pays and the app/browser dies, loses network, or the tab closes before `verify` , Razorpay captured the money while the `Payment` row stays `PENDING` and the fee stays unpaid. Schools see disputes and manual reconciliation. Double-submit safety is good; lost-callback safety is absent.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** reconciliation designed as client-driven only.
- **Recommended fix:** add `app/api/payments/razorpay/webhook/route.js` (raw body, `X-Razorpay-Signature` HMAC with a per-school webhook secret, handle `payment.captured` / `order.paid`), reusing the existing idempotent finalize logic from `verifyRazorpayPayment` . Add a nightly reconcile of PENDING payments older than 30 min via Razorpay `orders.fetchPayments` .
- **Validation:** replay the same webhook twice -> one SUCCESS payment, fee paid once; simulate closing the tab -> fee becomes PAID from the webhook.
- **Effort:** M **Dependencies:** per-school webhook secret storage next to `razorpayKeySecretCipher` .

ARC-003 - `middleware.js` uses the deprecated file convention

- **Category:** Backend / Framework
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Affected component:** `middleware.js`

- **Evidence:** `node_modules/next/dist/docs/01-app/03-api-reference/03-file-conventions/proxy.md:11` (deprecated, renamed `proxy`); `middleware.js:75` exports `middleware` and a `config.matcher`.
- **Impact:** still functional in 16.3.2 (verified only by the existence of a running dev log; not tested) but will break on a future major; the Edge-runtime assumption in the file's comment (line 3-5) no longer matches Proxy's Node.js default.
- **Reproduction / verification:** code/doc-based.
- **Root cause:** project predates Next 16 rename.
- **Recommended fix:** `npx @next/codemod@canary middleware-to-proxy .` (docs: `version-16.md` codemods) when convenient; rename function to `proxy`.
- **Validation:** redirects for `/dashboard` without cookie still work.
- **Effort:** S **Dependencies:** none.

ARC-004 - No error boundaries (`error.jsx`, `global-error.jsx`)

- **Category:** Frontend
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** `find app -name error.jsx -o -name global-error.jsx` returned nothing; only `app/not-found.jsx` exists. 40 `loading.jsx` files exist.
- **Impact:** any thrown error in a Server Component (e.g. a Prisma timeout inside `app/dashboard/layout.jsx`, which awaits several DB reads) shows Next's default error page with no recovery link and no place to hook error reporting.
- **Reproduction / verification:** code-based.
- **Root cause:** never added.
- **Recommended fix:** add `app/error.jsx` and `app/global-error.jsx` (client components with a retry button) and ship the error to the logging solution chosen in OPS-007.
- **Validation:** throw inside a page in dev and confirm the boundary renders.
- **Effort:** S **Dependencies:** OPS-007.

ARC-005 - Dead schema/code: `GeneratedReportJob`, three seed-data libs, eight components

- **Category:** Other
- **Severity:** P3 **Confidence:** High **Status:** Confirmed (reference grep)
- **Evidence:** no reference to `generatedReportJob` in `app/` or `lib/`; `lib/{classSeedData,studentSeedData,teacherSeedData}.js` (1,206 lines total) not imported anywhere; unreferenced components: `academics/ComingSoonCard`, `academics/TimeTablePeriodModal`, `dashboard/{ClassOverviewSection,RecentActivityCard,RecentStudentsTable,SmartAlertsCard,StudentOverviewCard,WelcomeBanner}`.
- **Impact:** noise, misleading status (a "report generation job" table suggests a feature that does not exist), bundle-agnostic but review cost.
- **Reproduction / verification:** `grep -rlE "[/'\"]<name>['\"]"` over `app lib components prisma`.
- **Root cause:** progressive migration from in-memory to Postgres left leftovers.
- **Recommended fix:** delete after a human confirms; drop the model in a real migration (see OPS-003).
- **Validation:** `npx eslint .` and a build still pass.
- **Effort:** S **Dependencies:** OPS-003.

11. Environment-variable inventory (names only)

Names found by `grep process.env.` in `app lib components middleware.js prisma` plus `env()` in `schema.prisma`. "Present in local .env" means the NAME exists in the local `.env` (values not read).

Variable	Used in	Validated at startup?	Failure mode
`DATABASE_URL`	`prisma/schema.prisma:20`	No (Prisma fails at first query)	Runtime 500
`JWT_SECRET`	`lib/auth/jwt.js`	No; throws lazily on first sign/verify; `verifySessionToken` swallows the throw and returns null (`jwt.js:24-26`), so a missing secret silently logs everybody out	Silent auth failure
`ENCRYPTION_KEY`	`lib/auth/encryption.js` (64-hex check, lazy)	Lazy	Payment setup/checkout throws
`CRON_SECRET`	both cron routes	**Not set in local .env**; optional by design (fail-open)	Cron endpoints open (OPS-001)
`SUPER_ADMIN_SEED_PASSWORD`	`prisma/seed.js`	n/a (random fallback printed to console)	
`NEXT_PUBLIC_APP_URL`	`lib/auth/...` / emails	No	Broken links in emails
`SMTP_HOST/PORT/USER/PASS/FROM/SECURE`	`lib/email.js`	No; unset => console log transport	Mail silently not sent
`R2_ACCOUNT_ID/ACCESS_KEY_ID/SECRET_ACCESS_KEY/BUCKET_NAME/PUBLIC_URL`	`lib/storage.js`	No	Upload/delete fail at runtime (delete swallowed)
`PUSHER_APP_ID/KEY/SECRET/CLUSTER`	`lib/pusher.js`	No	Best-effort failures
`NEXT_PUBLIC_PUSHER_KEY/CLUSTER`	`lib/pusherClient.js`	No (build-time inlined)	Realtime off
`NODE_ENV`	`lib/auth/session.js` (cookie `secure`)	Framework	

Razorpay keys are NOT environment variables: they are per-school in the DB (encrypted). No `.env.example` exists in ADMIN. No schema-based env validation exists anywhere.

Positive observations

- Per-school Razorpay keys with AES-256-GCM at rest and constant-time signature compare (`lib/razorpay.js:46-54`).
- `Payment.razorpayPaymentId` / `razorpayOrderId` unique constraints plus transactional update.
- Auth guard keyword coverage is high (177 of 180 handlers).
- Clean `lib/*Constants.js` split to keep client bundles free of server code.
- `.env*`, `*.pem`, firebase admin JSON and keystore are git-ignored; no secret files are tracked in ADMIN (verified with `git ls-files`).

Limitations / not assessed

- Runtime behaviour was not exercised; `localhost:3000` GET probes were not made in this report.
- R2 bucket ACL, Vercel project settings, production env values, and database size/indexes not accessible.
- Circular-import detection tool not run (no madge installed; avoided installing).
- Feature statuses marked Fully mean "path exists end to end in code", not "correct".

02 — Frontend Engineering and UI/UX Audit (Admin web app)

Auditor: A2. Scope: ADMIN `components/` (283 .jsx), `app/**/page.jsx` (70), layouts, loading/error files, `lib/api.js`, `lib/hooks/*`. Code-based review; **no visual/browser testing was performed** — all UX findings are "code-based, not visually verified".

Method

- Listed route groups and loading/error/not-found files (glob). Ran `npx eslint components app` (read-only, JSON output) and extracted rule counts.
- Grepped all of `app/ components/ lib/` for `dangerouslySetInnerHTML`, `innerHTML`, `document.write`, `window.open`, `router.push(<data>)`, `localStorage`, `NEXT_PUBLIC_*`, `AbortController`, `router.refresh`, `aria-*`.
- Read in full: `components/Modal.jsx`, `lib/hooks/useUrlSync.js`, `lib/hooks/usePagedResult.js`, `components/fees/RazorpayPayButton.jsx` (payment path), `components/fees/CollectFeeModal.jsx`, `printReportCard.js`; large parts of `StudentFeeDetailPanel.jsx`, `printMonthlyReport.js`; head of `lib/api.js`, `PageLoader.jsx`, DropdownMenu/Dropdown (top), Sidebar mobile classes. Everything else was sampled by grep only (about 15 of 283 components read in depth).

Summary table

ID	Sev	Conf	Title	Component
FE-001	P1	Medium	Online-payment finalisation depends only on the browser callback; no Razorpay webhook route	fees / Razorpay
FE-002	P2	High	No `error.jsx` / `global-error.jsx` anywhere in the App Router tree	app routing
FE-003	P2	High	`npx eslint` reports 55 errors incl. 47 `set-state-in-effect`	many components
FE-004	P2	Medium	No global 401/expired-session handling in the client fetch layer	lib/api.js, explorers
UX-001	P2	High	Modal has no dialog semantics, focus trap or accessible close; backdrop click discards forms	Modal.jsx
UX-002	P3	High	Custom dropdowns/menus: no keyboard or ARIA support; 2 `aria-label`s in the whole UI	Dropdown, DropdownMenu
UX-003	P3	High	Mixed loaders: 11 route `loading.jsx` use full-screen `PageLoader` vs shimmer skeletons	loading.jsx
UX-004	P3	Medium	Unsaved-changes protection exists in one screen only	forms
UX-005	P3	Medium	Pagination pattern inconsistent (fixed bottom bar vs inline, hand-copied bars)	list pages
FE-005	P3	High	Duplicate uncached server fetch in `generateMetadata` + page	sections/[sectionId], exams/[id]/verify
FE-006	P3	Medium	Filter change may trigger RSC refetch and client fetch; 43 files call `router.refresh()`	useUrlSync, explorers
FE-007	P3	Medium	No request cancellation (`AbortController` in 0 files)	components
FE-008	P4	High	Notification `link` passed straight to `router.push`	NotificationBell, ParentTopbar

ID	Sev	Conf	Title	Component
FE-009	P3	Medium	Very large components (1,023 / 739 / 737 / 687 / 680 lines)	maintainability

Findings

FE-001 — Online-payment finalisation depends only on the browser callback; no Razorpay webhook route

- **Category:** Frontend / Backend **Severity:** P1 **Confidence:** Medium **Status:** Confirmed (missing control); consequence is code-based
- **Affected component:** Fee payment via Razorpay (web `RazorpayPayButton`, mobile `ParentFeesScreen`)
- **Evidence:** `components/fees/RazorpayPayButton.jsx:63-80` — the payment is recorded only inside Razorpay's `handler`, which calls `verifyRazorpayPayment`; on failure it only shows `err.message`. `find app/api -ipath '*webhook*'` returns nothing and no file references `x-razorpay-signature`. Mobile `schoolapp360/src/features/parent/ParentFeesScreen.js:149-190` uses the same pattern; a verify failure is shown as "Payment failed" even though Razorpay has already captured the money.
- **Impact:** If the parent closes the tab/app, loses network or the verify call 5xx's after the card is charged, money is captured but the fee stays unpaid in the system; reconciliation is manual and the parent, told "failed", may pay twice.
- **Reproduction / verification:** code-based; not executed. Server-side idempotency of create-order/verify not reviewed here (backend auditor).
- **Root cause:** No server-to-server confirmation path (`payment.captured` webhook) and no client "confirming" recovery state.
- **Recommended fix:** Add `app/api/payments/razorpay/webhook/route.js` that verifies the webhook signature on the raw body and records the payment idempotently using the same function as verify. On client verify failure show "Payment may have been received — do not pay again; refresh in a minute".
- **Validation:** Simulate verify failure after a test-mode payment; the webhook still marks the fee paid exactly once.
- **Effort:** M **Dependencies:** MOB-005; backend auditor's payment findings.

FE-002 — No `error.jsx` / `global-error.jsx` anywhere

- **Category:** Frontend **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Affected component:** All route groups (`/dashboard`, `/parent`, `/super-admin`, auth pages, `/setup`)
- **Evidence:** `find app -name error.jsx -o -name global-error.jsx` returns nothing; only `app/not-found.jsx` exists. Most dashboard pages are async Server Components that call DB-backed lib functions directly (e.g. `app/dashboard/classes/page.jsx:12-19`).
- **Impact:** Any thrown DB/lib error during server render shows Next's generic error screen with no in-app navigation or retry.
- **Reproduction / verification:** code-based (file listing); not executed.
- **Root cause:** Only `loading.jsx` was rolled out; error boundaries never added.
- **Recommended fix:** Add `app/dashboard/error.jsx`, `app/parent/error.jsx`, `app/super-admin/error.jsx` and `app/global-error.jsx` (client components with a `reset()` button, no error text shown). Check `node_modules/next/dist/docs/` for this Next version's conventions first.
- **Validation:** Temporarily throw in one page in dev; boundary renders with Retry.
- **Effort:** S **Dependencies:** none.
- Loading coverage: `dashboard/loading.jsx`, `parent/loading.jsx`, `super-admin/loading.jsx` exist as group fallbacks plus about 40 per-route files; `login`, `forgot-password`, `set-password`, `setup` have none (acceptable).

FE-003 — ESLint reports 55 errors, 47 of them `react-hooks/set-state-in-effect`

- **Category:** Frontend **Severity:** P2 **Confidence:** High **Status:** Confirmed (ran `npx eslint components app`)
- **Affected component:** files below
- **Evidence:** rule counts — `set-state-in-effect` 47 (E), `purity` 4 (E), `immutability` 1 (E), `preserve-manual-memoization` 1 (E), `no-html-link-for-pages` 2 (E), `react/no-unescaped-entities` 42 (E), `no-img-element` 17 (W), `incompatible-library` 13 (W). set-state-in-effect sites (file:line): academics/SubjectFormModal:22, TimeTableClient:174,215, TimeTablePeriodModal:25; attendance/DailyAttendanceBoard:145, RemarkModal:13; classes/AssignClassTeacherModal:33, AssignSectionTeacherModal:29, SectionFormModal:44, SubjectsModal:17; dashboard/NotificationBell:52, TeacherNotificationBell:69; exams/EffectiveResultModal:15, ExamAuditLogModal:53, ExamEnrollmentModal:18, ExamVerificationClient:98, PrintMarksheetClient:38; fees/CollectFeeModal:44, DiscountModal:50, FeeStructureFormModal:437, GenerateFeesModal:22, PaymentLedgerModal:29, PaymentsExplorer:38, StandingDiscountModal:47, StudentFeeDetailPanel:180,201, StudentFeesExplorer:165; marksEntry/MarksEntryClient:22,38,48, MarksEntrySheet:68,90; parent/ParentFeesView:22, ParentTopbar:83; performance/BulkImportModal:64, MonthlyDashboardExplorer:72, SubjectTestsExplorer:50, YearlyDashboardExplorer:78; students/PromoteStudentsExplorer:82, SetPortalAccessDialog:30, StudentAttendanceTab:130, StudentFeesTab:33; superadmin/AdminPermissionsModal:17, CreateAdminModal:26, EditSchoolModal:42, ResetPasswordDialog:15; teachers/ResetPasswordDialog:16.
- **Impact:** Extra render passes and possible flicker (modals reset form state in an effect after open, e.g. `CollectFeeModal.jsx:44`); the lint baseline is red and there is no CI to gate it. Mostly maintainability/perf, not a demonstrated correctness bug.
- **Reproduction / verification:** `npx eslint components app` (read-only).
- **Root cause:** "Reset on open" done with `useEffect` +setState rather than remounting the form with a `key` or deriving state.
- **Recommended fix:** Render modal bodies only while open with `key={fee.id}` so state initialises from props; fix the 4 `purity` errors (impure calls in render) first.
- **Validation:** eslint error count falls; modals behave the same.
- **Effort:** M **Dependencies:** none.

FE-004 — No global 401/expired-session handling in the client fetch layer

- **Category:** Frontend **Severity:** P2 **Confidence:** Medium **Status:** Suspected
- **Affected component:** `lib/api.js` and ad-hoc `fetch` callers
- **Evidence:** `lib/api.js` helpers follow `if (!res.ok) { throw new Error(error.error || '...') }` (e.g. lines 1-50) with no status inspection; grep for `401` under `components/`, `lib/api.js`, `lib/hooks`, non-API `app/` finds only comments. Middleware behaviour for expired sessions on API calls was not assessed.
- **Impact:** After the session JWT expires (or a role is revoked) mid-session each action shows a generic "Failed to ..." message instead of redirecting to login.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Each helper re-implements error handling; there is no shared `apiFetch`.
- **Recommended fix:** Small `apiFetch(url, opts)` wrapper: on 401 `window.location.assign('/login')`, on 403 throw a "no permission" error; helpers call it.
- **Validation:** Delete the cookie in dev and click an action; redirected to login once.
- **Effort:** M **Dependencies:** none.

UX-001 — Modal lacks dialog semantics and focus management; backdrop click discards forms

- **Category:** UX / accessibility **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based, not visually verified)

- **Affected component:** `components/Modal.jsx` (used by nearly all form modals)
- **Evidence:** `Modal.jsx:27` Escape closes; `:37` backdrop `onClick={onClose}`; `:56-58` close button is icon-only with no `aria-label`; no `role="dialog"` / `aria-modal` /focus trap (`grep aria-modal|role="dialog" components` returns 0). Escape/backdrop also close while a submit is in flight.
- **Impact:** Keyboard and screen-reader users cannot use modals reliably; an accidental backdrop click on a long form loses all input.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Minimal shared modal.
- **Recommended fix:** `role="dialog" aria-modal="true" aria-labelledby`, `aria-label="Close"` on the X, focus first field on open and restore on close, optional `dismissOnBackdrop={false}` for forms (or native `<dialog>.showModal()`, which provides focus trap and Escape).
- **Validation:** Tab cycles within the modal; a screen reader announces the title.
- **Effort:** S **Dependencies:** none.

UX-002 — Custom dropdowns/menus have no keyboard or ARIA support

- **Category:** UX / accessibility **Severity:** P3 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `components/Dropdown.jsx`, `components/DropdownMenu.jsx`
- **Evidence:** both open on click and close on outside `mousedown`; no `onKeyDown`, `role=listbox/menu`, `aria-expanded` (DropdownMenu.jsx fully read to line 60, Dropdown.jsx partly). Across `components/` and `app/` only 3 files contain any `aria-` attribute and there are 2 `aria-label`s against 289 `<button` occurrences (many icon-only).
- **Impact:** Keyboard-only and assistive-tech users cannot operate class/section/status selectors in core workflows.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Hand-rolled portal widgets without an a11y pass.
- **Recommended fix:** Add Arrow/Enter/Escape handling and `aria-haspopup` / `aria-expanded`; `aria-label` on icon buttons; use native `<select>` for simple lists.
- **Validation:** Operate the dropdowns with keyboard only.
- **Effort:** M **Dependencies:** UX-001.

UX-003 — Loader inconsistency: PageLoader spinner vs shimmer skeletons

- **Category:** UX **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Affected component:** route `loading.jsx` files
- **Evidence:** `components/PageLoader.jsx` (full-screen fixed overlay spinner) is used by `app/dashboard/academics/{calendar,subjects,timetable}/loading.jsx`, `attendance/loading.jsx`, `exams/[id]/loading.jsx`, `fees/loading.jsx`, `leave/loading.jsx`, `dashboard/loading.jsx`, `profile/loading.jsx`, `settings/loading.jsx`, `app/super-admin/loading.jsx`. Redesigned areas (parent portal, classes/homework/notices/sessions) use skeletons per PageLoader's own header comment.
- **Impact:** Visible jump between spinner overlay and layout-matching skeletons; `dashboard/loading.jsx` is also the fallback for any dashboard route without its own file.
- **Reproduction / verification:** code-based, not visually verified.
- **Root cause:** Redesign rolled out incrementally.
- **Recommended fix:** Convert to skeleton loading files, starting with dashboard home, fees, attendance.
- **Validation:** Navigate between those routes; consistent placeholders.
- **Effort:** M **Dependencies:** none.

UX-004 — Unsaved-changes protection only in the timetable editor

- **Category:** UX **Severity:** P3 **Confidence:** Medium **Status:** Confirmed (grep-based)

- **Affected component:** marks entry, attendance, long forms
- **Evidence:** `components/academics/TimeTableClient.jsx:196,220` (`isDirty` + `window.confirm`). `grep unsaved|beforeunload` finds nothing else; attendance `isDirty` only enables buttons (`AttendanceActionBar.jsx:19`). `MarksEntrySheet`, `StudentForm` (737 lines), `TeacherForm` (680) have no leave-guard.
- **Impact:** Teachers lose entered marks/attendance on accidental navigation or refresh.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Not implemented outside one screen.
- **Recommended fix:** Small `useBeforeUnload(isDirty)` hook for marks entry and attendance.
- **Validation:** Edit, then refresh: browser prompts.
- **Effort:** S **Dependencies:** UX-001.

UX-005 — Pagination pattern inconsistent

- **Category:** UX **Severity:** P3 **Confidence:** Medium **Status:** Confirmed (code-based)
- **Affected component:** list pages
- **Evidence:** `FixedPaginationBar.jsx:16` (fixed bottom, `lg:left-64`) is used by Subjects, Calendar, Classes, Exams, Payments, Homework, Notices and the parent lists; the same bar is hand-copied in `ExamDetailsClient.jsx:232`, `StaffAttendanceBoard.jsx:388`, `MarksEntrySheet.jsx:479` (z-20 vs z-30), while `Pagination.jsx` is inline.
- **Impact:** Inconsistent look; stacked bars can overlap content on small screens (not visually verified).
- **Reproduction / verification:** code-based.
- **Root cause:** Pattern copied rather than shared.
- **Recommended fix:** One shared fixed bar component; drop copies.
- **Validation:** Visual check at phone width.
- **Effort:** S **Dependencies:** none.

FE-005 — Duplicate uncached server fetch in `generateMetadata` and the page

- **Category:** Frontend / Performance **Severity:** P3 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `app/dashboard/classes/[id]/sections/[sectionId]/page.jsx`, `app/dashboard/exams/[id]/verify/page.jsx`
- **Evidence:** section page calls `getSectionDetail` at `:10` and `:19`; verify page calls `getExamById` at `:8` and `:15`. `lib/classes.js:223` and `lib/exams.js:156` are not wrapped in React `cache()`.
- **Impact:** Doubled DB query per navigation on these routes.
- **Reproduction / verification:** code-based.
- **Root cause:** Same getter used by metadata and render without memoisation.
- **Recommended fix:** Wrap the two getters with `cache` from `react`.
- **Validation:** One query per request in DB logs.
- **Effort:** S **Dependencies:** none.

FE-006 — Possible double fetching on filter change; heavy `router.refresh()` use

- **Category:** Frontend / Performance **Severity:** P3 **Confidence:** Medium **Status:** Suspected
- **Affected component:** `lib/hooks/useUrlSync.js`, `usePagedResult.js`, explorers
- **Evidence:** `useUrlSync.js:24-33` does debounced (300 ms) `router.replace` on each change, which re-renders the server page (RSC refetch); `usePagedResult.js:36-52` separately calls client `fetchPage()` for the same key. 43 files call `router.refresh()`.
- **Impact:** Two DB round-trips per filter change on Homework/Notices-style explorers.
- **Reproduction / verification:** code-based; confirm in the browser Network tab.
- **Root cause:** URL sync and client fetching both active.

- **Recommended fix:** Pick one source of data per explorer (client fetch + `history.replaceState`, or server only).
- **Validation:** One request per filter change.
- **Effort:** M **Dependencies:** none.

FE-007 — No request cancellation

- **Category:** Frontend **Severity:** P3 **Confidence:** Medium **Status:** Confirmed
- **Affected component:** client components that fetch
- **Evidence:** `AbortController` appears in 0 files under `components/ app/ lib/`. `usePagedResult.js:41-52` guards with a `cancelled` flag (stale results cannot overwrite). Other effects were not individually audited.
- **Impact:** Wasted requests; possible stale overwrites where no flag exists (not enumerated).
- **Reproduction / verification:** grep-based.
- **Root cause:** Not part of the fetch helpers.
- **Recommended fix:** Optional `signal` argument in `lib/api.js` helpers where effects call them.
- **Validation:** Rapid filter switching leaves latest result.
- **Effort:** S-M **Dependencies:** FE-004.

FE-008 — Notification `link` passed to `router.push`

- **Category:** Frontend security **Severity:** P4 **Confidence:** High **Status:** Confirmed (low risk)
- **Affected component:** `components/dashboard/NotificationBell.jsx:102`, `components/parent/ParentTopbar.jsx:102`
- **Evidence:** `router.push(item.link)`. Links are server-generated internal paths (`lib/examMarks.js:269`, `lib/notices.js:551`, `lib/support.js:194`); no user-controlled source of `link` was found. `TeacherNotificationBell.jsx` maps links to web routes instead of trusting them.
- **Impact:** None demonstrated; would matter only if a future writer stores an external URL.
- **Reproduction / verification:** code-based.
- **Root cause:** No client-side validation of the stored link.
- **Recommended fix:** Guard with `link.startsWith('/') && !link.startsWith('//')`.
- **Validation:** External link ignored.
- **Effort:** S **Dependencies:** none.

FE-009 — Very large components

- **Category:** Maintainability **Severity:** P3 **Confidence:** Medium **Status:** Confirmed
- **Affected component:** `TimeTableClient.jsx` 1,023 lines, `RolesPermissionsExplorer.jsx` 739, `StudentForm.jsx` 737, `FeeStructureFormModal.jsx` 687, `TeacherForm.jsx` 680
- **Evidence:** `wc -l` on components.
- **Impact:** Hard to review; contributes to the effect-state issues (FE-003).
- **Reproduction / verification:** `wc -l`.
- **Root cause:** Organic growth.
- **Recommended fix:** Split opportunistically when touched; no big-bang refactor.
- **Validation:** n/a.
- **Effort:** L **Dependencies:** FE-003.

Workflow review (structure-level, not executed)

- Add student: one 737-line `StudentForm`, no leave-guard (UX-004).
- Record attendance: `DailyAttendanceBoard` + sticky `AttendanceActionBar` (Save disabled unless dirty).
- Enter marks: `MarksEntryClient` / `MarksEntrySheet` with 5 setState-in-effect sites and a fixed bottom bar.

- Collect fees: `CollectFeeModal` uses react-hook-form `isSubmitting` to disable submit; client amount cap plus (presumably) server check. Razorpay button disables via `isProcessing`; see FE-001 for the post-payment gap.
- Publish results, send notice, assign teacher: shared Modal (UX-001). Excess steps/confusing forms were not measured.

Frontend security observations

- No `dangerouslySetInnerHTML` / `innerHTML`; no `localStorage` / `sessionStorage` use; `NEXT_PUBLIC_*` names are `NEXT_PUBLIC_APP_URL`, `NEXT_PUBLIC_PUSHER_KEY`, `NEXT_PUBLIC_PUSHER_CLUSTER` (non-secret by design).
- Print/receipt HTML via `win.document.write` (`printReportCard.js:51`, `StudentFeeDetailPanel.jsx:43`, `printMonthlyReport.js:224,254`, `printYearlyReport.js:251,276`): user-derived strings pass through `escapeHtml` (escapes `& < > " '`); remaining unescaped interpolations are numeric fields. Print windows load Google Fonts (minor privacy note).
- Sampled server pages call `blockIfAccountant()/blockIfTeacher()` (e.g. `classes/page.jsx:12`), so access is not purely hidden-button; API enforcement belongs to other auditors.

Positive observations

- Consistent HTML escaping in all four print generators; no HTML sinks.
- `usePagedResult` discards stale responses and handles last-page deletion.
- Forms use `isSubmitting`; sticky action bars disable while saving.
- Skeleton `loading.jsx` on the parent portal and redesigned lists; mobile sidebar is an overlay drawer with `lg:` breakpoints (`Sidebar.jsx:415-422`); charts set `responsive: true, maintainAspectRatio: false`.
- `app/not-found.jsx` present.

Limitations / not assessed

- No browser or visual testing: contrast, touch-target size, table overflow on phones, chart rendering were not assessed beyond class names.
- About 15 of 283 components read in full. Not exhaustively reviewed: per-effect race analysis of the 47 flagged effects, necessity of `"use client"` (189 of 283 component files are client components), serialisation of large server data into client props, and Suspense wrapping for each of the 20 `useSearchParams` users.
- Server data-fetch waterfalls: 10 pages with 3+ awaits and no `Promise.all` were seen; those inspected were dependent chains (auth, record, list), so only duplicate fetches are reported.

03 — Backend and API audit (auditor A3)

Prefixes: `API` (endpoint correctness), `BIZ` (business-logic integrity). Scorecard area: "Backend and APIs". Audit date 2026-10-10, current working tree, code-based plus read-only `GET` probes against `http://localhost:3000` (no cookie, no POST/PUT/PATCH/DELETE, no `/api/cron/*`).

Scope

ADMIN `app/api/**/route.js` (180 files, 225 exported handlers), `app/actions/auth.js`, `lib/*.js` business logic (fees, students, attendance, exams, marks, results, notices, support, storage, iam, rbac, expoPush, pusher, razorpay), `lib/schemas.js`, `middleware.js`, `vercel.json`.

Method and sampling

- Inventory: glob of all 180 `route.js`, scripted scan for methods and auth tokens (`requirePermission`, `getCurrentUserInfo`, `requireParent`, `requireTeacher`, `requireSchoolAdmin`, `requireSuperAdmin`, `CRON_SECRET`). Every file whose scan returned NO auth token was then read in full.
- **Fully read (code, line by line): 111 of 180 route files** (all fees, payments, attendance, exams core, students core, promote, support, every `*upload-url`, notices, leaves, homework, sessions, users, roles, schools POST, parent/teacher samples, cron). **Remaining ~69 files** (e.g. most `performance/*`, `school/*`, `classes/*/sections/*`, `admins/*`, `parent/*` read-only lists, `teacher/*` self-service) were classified from the token scan plus the pattern of the same handler family; they are reported as "gate seen, body not read".
- Libs traced route to lib to prisma: `lib/fees.js` (generation, discount, collect, bulk-collect, Razorpay order/verify), `lib/students.js` (add/promote/retain/update/delete), `lib/attendance.js`, `lib/teacherAttendance.js`, `lib/examMarks.js`, `lib/examResults.js`, `lib/exams.js` (status), `lib/notices.js` (fan-out), `lib/support.js`, `lib/storage.js`, `lib/iam.js`, `lib/rbac.js`, `lib/currentUser.js`, `lib/school.js`, `lib/auth/schoolContext.js`.
- Verified live (GET only, no cookie): `/api/support/tickets`, `/api/subjects`, `/api/iam/me`, `/api/classes/sections-map`, `/api/exams`, `/api/fees/students/<id>/ledger`, `/api/notices`, `/api/homework`, `/api/students`, `/api/timetable`, plus templates. Response bodies were not copied here.
- Not verified by execution: every mutating-handler finding (code-based only). No DB queries were run.

Summary table

ID	Sev	Conf	Title	Component
API-001	P0	High	Session-less requests are served as a global "dashboard Teacher" actor; unauthenticated GET returns cross-school support tickets	`lib/iam.js`, `lib/currentUser.js`, support/exams/homework/notices routes
API-002	P1	High	`POST /api/schools` has no authentication and repoints the global active-school singleton	`app/api/schools/route.js`
API-003	P1	High	Five handlers have no auth check on any path (fee ledger, create-order, verify, attendance by student, subjects GET)	see inventory
BIZ-001	P1	High	Fee-generation cron processes one school only (global singleton), and fails open if `CRON_SECRET` is unset	`lib/fees.js:583`, `app/api/cron/generate-fees`
BIZ-002	P1	High	No Razorpay webhook; verification relies on the browser callback; abandoned checkouts lose captured payments	`lib/fees.js:1147-1265`
BIZ-003	P1	High	Online/manual payment races and overpayment: stale read, absolute write, many PENDING orders per fee, no cap	`lib/fees.js:1025-1265`
BIZ-004	P1	High	Attendance lock rules are disabled in code; `markedBy` and `records` are client-controlled	`lib/attendance.js:103-110`, `app/api/attendance/route.js`

ID	Sev	Conf	Title	Component
API-004	P1	High	Roles/permissions are global (not per tenant); any holder of `permissions.manage` edits roles for all schools	`app/api/roles/[id]/permissions`, `prisma Role`
BIZ-005	P2	High	No refund/reversal path; no collector or actor recorded on payments and discounts	fees module
BIZ-006	P2	High	Marks edits after approval/publish do not invalidate or recompute results; marks save is non-transactional	`lib/examMarks.js:164-258`, `lib/examResults.js`
BIZ-007	P2	High	Attendance reports key history on the student's CURRENT class/section	`lib/attendance.js:190-196`
BIZ-008	P2	Medium	Promotion: promote and retain run as parallel transactions; no status/target validation; no fee/dues handling	`app/api/students/promote`, `lib/students.js:738-875`
BIZ-009	P2	Medium	Notification/push fan-out is fire-and-forget (un-awaited) on serverless; no retry or outbox	`lib/notices.js:542`, `lib/fees.js:1052`
API-005	P2	High	No server-side schema validation; client-only zod; malformed JSON gives 500	all POST/PUT routes
API-006	P2	High	Unbounded list responses and unbounded request arrays	students, users, notices, bulk-collect, bulk-import
API-007	P2	Medium	Upload presign does not bind size; free-form `attachmentUrl` / `photoUrl` stored; cleanup cron can delete another tenant's R2 object	upload-url routes, `lib/support.js:229`
API-008	P2	High	Rate limiting exists only on logins and is in-memory per instance	`lib/auth/loginRateLimit.js`
API-009	P3	High	`err.message` (incl. raw Prisma/SDK text) returned to clients in ~97 places	all routes
API-010	P3	High	Temp passwords generated with `Math.random` and returned in the response	`app/api/users/route.js:17`, `lib/admins.js:6`, `lib/parentAccounts.js:6`
BIZ-010	P3	Medium	Exam status has no transition rules; re-generating results silently unpublishes	`lib/exams.js:279`, `lib/examResults.js:207`
API-011	P3	Medium	Sequential awaits / N+1 / full-school loads in fee generation, bulk import, teacher student list	`lib/fees.js:437`, `lib/students.js:540`

Endpoint inventory (180 files, 225 handlers)

Auth gate = what the handler calls first. "Perm" = permission gate. "Scoped" = tenant/ownership scoping inside the lib (by `schoolId` from `resolveSchoolId()`, which prefers the session claim and otherwise falls back to the global singleton, see API-001/SEC).

Area	Files / handlers	Auth gate	Permission gate	Notes
auth	4 / 4	none by design (login); `getSession` on parent/switch	in-memory rate limit on 3 logins	`await request.json()` before try
students	15 / 22	`requirePermission` (most); `getCurrentUserInfo` role check on `GET /students`, `/paged`, `/print-details`	students.create/update/delete/view; Teacher extra block	`bulk-import/template` + `blank-template` have no auth (static xlsx, benign)
teachers	8 / 9	`requirePermission`; upload-urls `requireAnyPermission` / `requireTeacher`	teachers.*	

Area	Files / handlers	Auth gate	Permission gate	Notes
classes	10 / 11	`requirePermission('classes.manage')`; `sections-map` `getCurrentUserInfo`	classes.manage	any signed-in user may read sections-map
attendance	7 / 8	`requirePermission` + `getCurrentUserInfo` fallback; `requireTeacher` check-in; `student/[id]` none	attendance.student.*	`student/[id]` has no auth (API-003)
staff-attendance	1 / 2	`requirePermission` + Teacher block	attendance.teacher.*	
exams / marks / results	16 / 25	mix of `requirePermission` and bare `getCurrentUserInfo`; ownership enforced in lib (`assertCanAccessSchedule`, `assertCanManageSchedule`)	exams.manage, marks.update, results.publish/view	status, schedules, marks routes rely on lib-level role checks only
fees / payments / razorpay	18 / 22	`requirePermission` for admin; `getCurrentUserInfo` + Parent ownership for parent reads; **none** on ledger, create-order, verify	fees.view/create/update/collect	`fees.refund` permission exists, no refund endpoint
notices	5 / 8	`requirePermission` for writes; `getCurrentUserInfo` for reads	notices.*	
homework	3 / 5	`getCurrentUserInfo` (401 if null)	role scoping in lib	fixed in tree vs old toggle fallback
leaves	3 / 4	`requirePermission`	leave.apply/view/approve/reject	
calendar-events	2 / 4	`requireSchoolAdmin`	role only (no RBAC key)	scoped by schoolId in lib
timetable	2 / 4	`getCurrentUserInfo` / `requirePermission` (list)	class-teacher check for writes	
performance	21 / 23	`requirePermission` (all) + Teacher blocks	performanceReports.*	4 bodies read, rest by pattern
support (new)	6 / 7	`getCurrentUserInfo` + Parent block; `requireSuperAdmin` for status	none; tenancy via `actor.schoolId`	see API-001
parent	11 / 11	`requireParent` (all)	ownership via `actor.studentId` from session	good
teacher (mobile)	7 / 7	`requireTeacher` (all)	scoped by `teacherId`	good
sessions	4 / 4	`requirePermission('settings.update')`	settings.update	no transaction on activate (BIZ minor)
roles / permissions / users	3+1+3 / 5+1+4	`requirePermission`	roles., *permissions.manage, users.*	roles are global (API-004)
school (settings)	6 / 6	`requirePermission('settings.update')`	settings.update	
schools / admins / super-admin	7+5 / 9+5	`requireSuperAdmin` except **`POST /api/schools`**	role only	
cron	2 / 2	`CRON_SECRET` bearer, fail-open if env unset	none	
uploads (13 routes)	within areas	RBAC or role check per route	type allow-list + client-stated size	see API-007

Area	Files / handlers	Auth gate	Permission gate	Notes
misc (iam/me, notifications/register-token, print-marksheet, admin*, subjects)	8 / 11	`getCurrentUserInfo`; **subjects GET none**		

Handlers with NO authentication call on any path

#	Handler	File:line	Reads/writes	Cross-ref
1	`POST /api/schools`	`app/api/schools/route.js:21`	creates a School row, calls `setActiveSchoolContext`, creates and activates an academic session	SEC
2	`GET /api/fees/students/[id]/ledger`	`app/api/fees/students/[id]/ledger/route.js:5`	full payment history for any studentId in the resolved school	SEC
3	`POST /api/payments/razorpay/create-order`	`app/api/payments/razorpay/create-order/route.js:5`	creates a real Razorpay order and PENDING Payment for any studentFeeId	SEC
4	`POST /api/payments/razorpay/verify`	`app/api/payments/razorpay/verify/route.js:5`	marks a fee paid when signature valid	SEC
5	`GET /api/attendance/student/[id]`	`app/api/attendance/student/[id]/route.js:5`	attendance stats for any student	SEC
6	`GET /api/subjects`	`app/api/subjects/route.js:11`	lists subjects and runs `ensureSeeded` (a DB write) for the fallback school; confirmed 200 live with no cookie	SEC
7-8	`GET /api/students/bulk-import/template`, `/blank-template`	`.../template/route.js:182`, `.../blank-template/route.js:42`	static xlsx, benign	none

Handlers whose "auth" is only `getCurrentUserInfo()` non-null (and therefore inherit API-001): `exams` GET family, `exams/[id]/status`, `exams/[id]/schedules`, `exams/schedules/[scheduleId]/marks|route`, `homework*`, `notices GET/page/paged`, `print-marksheet*`, `timetable GET`, `classes/sections-map`, `support/*`, `iam/me`, `notifications/register-token`, `fees/student/[studentId]*`, `students GET/paged/print-details`. Role-only checks without ownership scoping: `calendar-events*` (`requireSchoolAdmin`, scoped in lib), `admin/profile*`.

Findings

API-001 — Session-less requests are served as a global "dashboard Teacher" actor

- **Category:** Backend (cross-ref SEC)
- **Severity:** P0 **Confidence:** High **Status:** Confirmed (live GET, code-confirmed)
- **Affected component:** `lib/iam.js` `getCurrentUserInfo`, `lib/currentUser.js`, ~25 handlers that treat a non-null actor as signed in
- **Evidence:** `lib/iam.js:199-210` falls back to `getDashboardRoleUser()` when there is no session; `lib/currentUser.js:9` keeps the role in `globalThis.__CURRENT_USER__`; every web login calls `setCurrentRole` (`app/actions/auth.js:92,111,138`), so the last web Teacher login makes every cookie-less request a Teacher. That fallback actor has no `schoolId`. Live, no cookie: `GET /api/iam/me` returned a Teacher user, `/api/exams`,

`/api/classes/sections-map`, `/api/subjects`, `/api/notices` returned 200, and `GET /api/support/tickets` returned ticket objects (including `ticketNumber`, `schoolId`) because `getSchoolTickets(undefined)` (`lib/support.js` where `{schoolId: undefined}`) applies no filter, i.e. tickets of ALL schools (`app/api/support/tickets/route.js`, `lib/support.js getSchoolTickets`).

- **Impact:** unauthenticated cross-tenant read of support tickets (subjects, messages, names, attachment URLs); unauthenticated access to every route that only checks `if (!actor)`. State depends on which role last logged in on that server instance, so it is intermittent in serverless but permanent on a long-lived instance.
- **Reproduction / verification:** `curl http://localhost:3000/api/support/tickets` with no cookie (done, GET only).
- **Root cause:** a demo "role preview" singleton left inside the production identity resolver.
- **Recommended fix:** remove the dashboard-toggle branch from `getCurrentUserInfo` (return `null` when `getSession()` is empty); delete `lib/currentUser.js` global state or confine it to a dev flag; in `getSchoolTickets` and similar helpers throw if `schoolId` is falsy.
- **Validation:** with no cookie, all `/api/*` except login return 401; add a smoke script asserting this.
- **Effort:** S–M **Dependencies:** API-003, SEC findings

API-002 — `POST /api/schools` is unauthenticated and flips global tenant context

- **Category:** Security/Backend
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code-based; not executed)
- **Evidence:** `app/api/schools/route.js:21-53` — no `requireSuperAdmin` (only the `GET` at line 7 has it). It calls `addSchoolFromWizard` (`lib/schools.js:120`, creates a School), then `setActiveSchoolContext(school)` (`lib/school.js`, mutates `globalThis.__ACTIVE_SCHOOL__`) and `addSession` / `setActiveSession`.
- **Impact:** anyone can create tenants (code collision probing, junk rows) and repoint the instance's fallback school, which affects the cron job, SuperAdmin sessions and API-001 actors for all later requests on that instance.
- **Recommended fix:** `const { error } = await requireSuperAdmin(); if (error) return error;` at the top; pass the new school id explicitly to session creation instead of mutating the singleton.
- **Validation:** unauthenticated POST returns 403. **Effort:** S **Dependencies:** API-001, BIZ-001

API-003 — Five handlers without any authentication check

- **Category:** Backend (cross-ref SEC)
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code; GET ledger/subjects confirmed live)
- **Evidence:** ledger `route.js:5-18` only blocks a Parent for a different child, never 401s on null; `create-order/route.js:11-17` and `verify/route.js:11-17` run the ownership check only when `actor?.role === 'Parent'`, otherwise proceed; `attendance/student/[id]/route.js:20-22` same pattern; `subjects/route.js:11`. Contrast the sibling `fees/student/[studentId]/route.js` which was fixed.
- **Impact:** payment history and attendance of any student readable without login; unauthenticated Razorpay order creation (creates PENDING Payment rows and live Razorpay orders: abuse/spam); unauthenticated verify.
- **Recommended fix:** use `requireParent` / `requirePermission('fees.view')` or at minimum `if (!actor) 401` plus ownership check for any non-admin role; add a shared `requireActor()` helper.
- **Validation:** unauthenticated call returns 401. **Effort:** S **Dependencies:** API-001

BIZ-001 — Fee generation cron serves one school and fails open

- **Category:** Backend
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code)
- **Evidence:** `lib/fees.js:583-607` `generateDueStudentFees` calls `getFeeStructures()` which resolves the school via `resolveSchoolId()` (`lib/fees.js:217`) with no session, i.e. the global singleton (`lib/school.js`, defaults to

a seed id). `app/api/cron/generate-fees/route.js:12-15` and `cleanup-support-tickets/route.js` only check the bearer when `process.env.CRON_SECRET` is set (fail-open).

- **Impact:** in a multi-school deployment only one school gets auto-generated fees; the rest silently never do. If the secret is missing the endpoints are publicly callable (mass generation / ticket purge).
- **Recommended fix:** loop `prisma.school.findMany({status:'Active'})` and pass `schoolId` into every lib call; return 500 when `CRON_SECRET` is unset.
- **Validation:** staging with two schools, cron yields fees for both; request without secret returns 401. **Effort:** M **Dependencies:** API-002

BIZ-002 — Razorpay: no webhook, browser-callback-only confirmation

- **Category:** Backend (financial)
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code; grep shows "webhook" only in comments)
- **Evidence:** no `webhook` route exists under `app/api`; `lib/fees.js:1186-1193` comment acknowledges it. `verifyRazorpayPayment` (`lib/fees.js:1194-1265`) is reached only from the client POST.
- **Impact:** if the parent's browser/app closes, loses network, or the callback fails after Razorpay captures money, the Payment row stays PENDING forever (no cleanup job, comment at `lib/fees.js:1142`) and the fee stays unpaid: money taken, school sees no payment. No reconciliation job either.
- **Handled:** HMAC-SHA256 signature check with per-school secret and `timingSafeEqual` (`lib/razorpay.js`), order must belong to the fee and school (`fees.js:1204`), amount decided server-side, `razorpayPaymentId` unique, duplicate verify short-circuits on `status === 'SUCCESS'`, writes in one `$transaction`.
- **Missing:** webhook (`payment.captured` / `order.paid`) with signature verification and idempotency; gateway-side amount fetch; stale PENDING expiry/reconciliation; refund webhook.
- **Recommended fix:** add `app/api/payments/razorpay/webhook/route.js` verifying `X-Razorpay-Signature` over the raw body with the school's webhook secret, resolving the Payment by `razorpayOrderId`, and reusing one shared `applySuccessfulPayment(paymentId)` that is idempotent and runs in a transaction with a conditional update (`updateMany where status:'PENDING'`).
- **Effort:** M **Dependencies:** BIZ-003

BIZ-003 — Payment races and overpayment

- **Category:** Backend (financial)
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code-based; not executed)
- **Evidence:** `collectManualPayment` (`lib/fees.js:1025-1081`) reads the fee, computes `newPaidAmount = fee.paidAmount + collectAmount` in JS and writes it as an absolute value; no row lock, no idempotency key. `verifyRazorpayPayment` (`fees.js:1239-1251`) does the same with `payment.amount`, never re-checking that the amount is still owed. `createRazorpayOrder` (`fees.js:1147`) creates a new PENDING order on every call with no reuse or cap.
- **Impact:** (1) two concurrent manual collections both pass the `<= remainingDue` check, create two Payment rows, but `paidAmount` is overwritten with one value (ledger sum greater than `paidAmount`); (2) a parent who opened two checkouts, or paid online after an admin collected cash, can have both verified: `paidAmount` exceeds payable, status PAID, no refund path (BIZ-005); (3) double-click on collect creates duplicate Payments.
- **Recommended fix:** use atomic updates: `prisma.studentFee.updateMany({ where: {id, schoolId, paidAmount: fee.paidAmount}, data:{ paidAmount: {increment: amt}}})` inside the transaction and abort on `count===0`; in verify re-compute remaining due and reject/flag overpay; reuse an existing PENDING order for the same fee; accept an `Idempotency-Key` on `/api/fees/collect`.
- **Validation:** concurrent test (two parallel POSTs) leaves `paidAmount === sum(SUCCESS payments)`. **Effort:** M

BIZ-004 — Attendance edit/lock rules disabled; client-controlled fields

- **Category:** Backend

- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code)
- **Evidence:** `lib/attendance.js:103-110` `ATTENDANCE_LOCK_ENABLED = false` makes `computeAttendanceAccess` always allow; the manual unlock (`manuallyUnlocked`, `attendance/[id]/lock`) therefore has no effect. `app/api/attendance/route.js` passes the raw body to `saveAttendance` (`lib/attendance.js:57`), which upserts `markedBy` (client supplied, `DailyAttendanceBoard.jsx:202`) and `records` (no schema check: statuses, studentIds belonging to the class, duplicates). Only Sunday is rejected; school calendar holidays and future dates are accepted.
- **Impact:** a Teacher can alter any past or future date for their class, forge `markedBy`, and write arbitrary JSON into `records`, corrupting reports. Concurrent submissions are last-write-wins via the `[school, session, date, class, section]` unique upsert (no duplicate rows, which is good).
- **Recommended fix:** turn the flag on (or delete the dead matrix and decide the policy), derive `markedBy` from the session, validate `records` against the roster and `ATTENDANCE_STATUSES`, reject future dates.
- **Effort:** S–M

API-004 — Roles and permissions are global, not per tenant

- **Category:** Security/Backend (cross-ref TEN)
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code)
- **Evidence:** `prisma Role` has `key @unique` and no `schoolId` (`schema.prisma:70-82`); `app/api/roles/[id]/permissions/route.js` replaces `RolePermission` rows of that single global role; `prisma/rbacPermissions.js:98` gives `PRINCIPAL = ALL_KEYS` which includes `permissions.manage` (`:70`).
- **Impact:** a Principal of school A can change what Admin/Teacher/Accountant may do in every other school (including granting themselves nothing more, but revoking fees access platform-wide, or giving Teachers `students.delete`).
- **Recommended fix:** restrict `permissions.manage` to SuperAdmin, or make roles per-school. **Effort:** S (restrict) / L (per-school)

BIZ-005 — No refund/reversal, thin financial audit trail

- **Category:** Backend (Missing control)
- **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Evidence:** permission `fees.refund` exists (`prisma/rbacPermissions.js:128`) but no handler uses it; `Payment` model (`schema.prisma:1427`) has no `collectedBy`, `reference`, `receiptNo`, `refundedAt`; `applyFeeDiscount` blocks discounts below paid amount ("process a refund first", `fees.js:870`) yet no refund exists. Discount apply/remove, fee delete and collections write no audit row (exams have `logExamAudit`, fees do not).
- **Impact:** wrong or duplicate collections cannot be reversed in-app; no answer to "who collected or discounted this".
- **Recommended fix:** add `Payment.collectedById/collectedByName`, a `FeeAuditLog` model written in the same transaction, and a `POST /api/fees/payments/[id]/reverse` gated by `fees.refund`. **Effort:** M–L
- Money handling: `Int` rupee columns (no floats), PERCENT discount rounded with `Math.round`, late fee computed on read (`lateFeeAmountFor`) so `payableAmount` drifts upward daily while `status` was set PAID earlier with a fixed `paidAmount` (BIZ-003 side effect); manual amounts like `10.5` would raise a Prisma Int error returned as 400 with raw message.

BIZ-006 — Marks and results integrity

- **Category:** Backend
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code)
- **Evidence:** `saveExamMarks` (`lib/examMarks.js:164-258`) for an admin overwrites `status` with `Draft` / `Submitted` on rows that were `Approved` / `Published` (`:247`) and does not touch `ExamResult`; results are only recomputed when an admin runs generate manually (`examResults.js:113`). Upserts run one by one without a transaction

(`:240-258`), so a failure leaves a partly saved sheet. `row.studentId` is not checked against the schedule roster. Max marks are checked at save time but `updateExamSchedule` can lower `maxMarks` / `passingMarks` later with no re-validation. `generateExamResults` re-upsert resets `status` to `Draft` (`:207`), un-publishing results parents already saw.

- **Handled well:** teacher scoping (`assertCanAccessSchedule`), teacher lock after Submit, Approve/Reject/Unlock admin-only with audit log (`logExamAudit`), negative/over-max checks, completeness check on submit, optional-subject enrollment honoured in result generation.
- **Missing:** retake/re-exam policy beyond a `ReExam` attendance value; automatic invalidation of published results on mark edits.
- **Recommended fix:** when a mark of a published exam changes, flag the student's `ExamResult` stale (or recompute) and keep status; wrap the row loop in `$transaction`; validate studentIds against the roster. **Effort:** M

BIZ-007 — Attendance history keyed by current class/section

- **Category:** Backend
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code)
- **Evidence:** `statusFromMap` (`lib/attendance.js:190-196`) looks up `${date}|${student.class}|${student.section}` using the student's CURRENT class/section for all historical dates; `relevantStudents` also filters on the current `academicSession`.
- **Impact:** after a section change or promotion within the same session view, earlier days vanish from the student's report; a student recorded in two blobs (old and new section the same day) is counted once and silently.
- **Recommended fix:** join against `StudentEnrollment` for the date range or scan all `records` blobs for the studentId. **Effort:** M

BIZ-008 — Student promotion/retention gaps

- **Category:** Backend
- **Severity:** P2 **Confidence:** Medium **Status:** Confirmed (code)
- **Evidence:** `app/api/students/promote/route.js` runs `promoteStudents` and `retainStudentsInSession` with `Promise.all` (separate transactions); a student id in both lists, or a conflict in one, leaves the other committed. `promoteStudents` (`lib/students.js:738`) does not require `status: 'Active'`, does not check the target class is above the source, and does nothing about unpaid fees of the old session or fee generation for the new one.
- **Handled well:** target session/class/section must exist, enrollment unique per `(studentId, academicSessionId)`, `StudentPromotionLog` row per action with actor, `StudentEnrollment` history, withdrawal via status (`TransferredOut/Inactive/Graduated`) updates enrollment and logs, delete blocked when fee/payment rows exist, admission number uniqueness via DB unique + P2002 (no race; the number is entered manually, not generated). `addStudent` silently skips the initial enrollment row if session/class are missing.
- **Recommended fix:** run the two operations sequentially in one transaction, filter `status: 'Active'`, call `generateFeesForNewStudent` after promotion. **Effort:** M

BIZ-009 — Fire-and-forget notifications

- **Category:** Backend
- **Severity:** P2 **Confidence:** Medium **Status:** Suspected (platform behaviour)
- **Evidence:** `notifyNoticeRecipients` is called without `await` (`lib/notices.js:542`), as are `notifyPaymentConfirmed` (`lib/fees.js:1052`, `:1130`), `createAdminNotification`. On Vercel the function can be frozen after the response is sent. Expo errors are only logged; `DeviceNotRegistered` tokens are pruned (good); no retry; support Pusher events use public channels with ticket ids only.
- **Recommended fix:** use `after()` / `waitUntil` (see `node_modules/next/dist/docs` for the current API) or await inside the request. **Effort:** S

API-005 — No server-side validation; malformed JSON gives 500

- **Category:** Backend
- **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Evidence:** `lib/schemas.js` (zod, 659 lines) is imported only by client components (e.g. `components/fees/CollectFeeModal.jsx`); no `app/api` route imports it. 110 `await request.json()` calls, 0 with `.catch`; 36 route files have no `try` at all, and in most others `request.json()` sits before the `try`. Validation is `if (!field)` checks. Content-Type is not checked. Mass assignment is limited: `buildStudentFields` / `fieldsFrom` whitelist fields (no raw body spread into prisma found; only server-built `...data` in `examMarks.js:255`, `examSchedules.js:227`).
- **Impact:** bad bodies return 500 with a stack in logs; numeric fields (`amount`, `marksObtained`, `maxMarks`) can be strings, NaN, fractional or negative until a lib check catches them.
- **Recommended fix:** a 10-line `parseBody(request, schema)` helper using the existing zod schemas (reuse `feeStructureSchema`, `manualPaymentSchema`, etc.) returning 400. **Effort:** M

API-006 — Unbounded lists and arrays

- **Category:** Backend/Performance
- **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Evidence:** `GET /api/students` returns the full decorated school (`app/api/students/route.js`), for Teachers it loads all students then filters in JS; `GET /api/users`, `GET /api/notices`, `GET /api/homework`, `GET /api/support/tickets`, `GET /api/subjects`, `getPaymentLedger`; `POST bulk-collect` `studentIds`, `students/bulk-import` `rows`, `exams/schedules/.../marks` `rows` and `fees/send-reminder` `studentIds` have no length cap. Paged variants exist for students, teachers, notices, homework, payments (page size capped at 50/100), but `payments` and `schools` do not cap `pageSize`.
- **Recommended fix:** cap array lengths (e.g. 500) and `pageSize`; migrate remaining list consumers to the paged routes. **Effort:** S–M

API-007 — Upload handling

- **Category:** Backend/Security
- **Severity:** P2 **Confidence:** Medium **Status:** Confirmed (code)
- **Evidence:** 13 `*upload-url` routes (list: students photo/document, teachers photo/document, school logo, admin/parent/teacher profile photo, notices, support). All require auth and allow-list `fileType`; keys are prefixed by `schoolId` (profile photos by user id) plus a sanitized name. But size is a client-stated `fileSize` only; `getPresignedUploadUrl` (`lib/storage.js`) signs `ContentType` but no `ContentLength`/size condition, so the real size is unlimited; no virus or magic-byte check; URLs are public (`R2_PUBLIC_URL`). `attachmentUrl` (support tickets/replies, `lib/support.js:111,130`) and `photoUrl` (self-profile PATCH) are stored without checking they point to this tenant's key prefix; `cleanupOldTickets` (`lib/support.js:229-232`) deletes whatever R2 key such a URL maps to.
- **Impact:** storage abuse; a school user can reference another school's R2 object in a ticket and have it deleted by the cron after 90 days; arbitrary external URL rendered as a link.
- **Recommended fix:** validate `attachmentUrl` / `photoUrl` start with `R2_PUBLIC_URL + '/<prefix>/<schoolId>/'`; use a presigned POST with `content-length-range`. **Effort:** S–M

API-008 — Rate limiting

- **Category:** Backend
- **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Evidence:** `lib/auth/loginRateLimit.js` is an in-memory `Map` (per instance, reset on cold start), used by login/reset routes only. No limit on create-order, upload-url, support tickets, bulk endpoints, or fee collection.

- **Recommended fix:** a shared store (Upstash/Postgres) for login; per-user caps on order creation and ticket creation. **Effort:** M

API-009 — Error message leakage

- **Category:** Backend
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** 97 `err.message` returns across routes with status 400 (e.g. `create-order/route.js` last line, `fees/collect`, `students/[id]`), so Prisma error text and SDK messages reach the client; wrong status for server faults (should be 500). The Razorpay handler also string-matches TypeErrors. No stack traces are returned.
- **Recommended fix:** a `toApiError(err)` mapper: known business errors (custom class) to 400, everything else to 500 with a generic message and server log. **Effort:** M

API-010 — Weak temp-password generation

- **Category:** Security (cross-ref AUTH)
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** `app/api/users/route.js:17-19`, `lib/admins.js:6-8`, `lib/parentAccounts.js:6-8` use `Math.random().toString(36).slice(2,8)` (about 31 bits, not cryptographic) and return the password in the JSON response (`users/route.js:97`).
- **Recommended fix:** `crypto.randomBytes(9).toString('base64url')`; force change at first login. **Effort:** S

BIZ-010 — Exam status transitions

- **Category:** Backend
- **Severity:** P3 **Confidence:** Medium **Status:** Confirmed
- **Evidence:** `setExamStatus` (`lib/exams.js:279`) accepts any of Draft/Published/Completed from admin with no source-state rule; `publishExamResults` forces `Completed`; regenerate resets results to Draft. **Fix:** explicit transition table. **Effort:** S

API-011 — Sequential awaits and full loads

- **Category:** Performance
- **Severity:** P3 **Confidence:** Medium **Status:** Confirmed
- **Evidence:** `generateStudentFees` loads every student then filters in JS and builds one transaction of N creates (`lib/fees.js:437-490`); the `@@unique([studentId, academicSession, term])` makes a cron/admin race abort the whole transaction with P2002 (500). `bulkCreateStudents` is sequential (each `addStudent` = several queries) and the route then runs `Promise.all` of fee generation for every imported student (`students/bulk-import/route.js`). `bulkCollectFullDue` and `applyStandingDiscountToPendingFees` loop with per-row updates.
- **Recommended fix:** query students with `where: {class, academicSession, status}`, use `createMany({skipDuplicates:true})`, bound concurrency. **Effort:** M

Critical flows: handled vs missing

- **Fees, generation:** handled: idempotent create-or-resync of PENDING, PAID never touched, per-term unique constraint, late fee snapshot per item, new-student auto-billing. Missing: multi-school cron (BIZ-001), concurrency (API-011).
- **Fees, collection:** handled: `fees.collect` permission, status validation, partial payments, one `$transaction`. Missing: idempotency, atomic increment, collector id, receipt numbering, refund (BIZ-003/005).
- **Razorpay:** handled: server-side amount, per-school keys (encrypted secret), signature with timing-safe compare, order/fee/school binding, duplicate verify returns existing state, unique `razorpayPaymentId`. Missing: webhook, reconciliation, expiry of PENDING orders, authentication on create-order/verify (API-003), overpay guard.

- **Attendance:** handled: unique upsert per class/day, teacher class-scope check, Sunday rule. Missing: lock rules (disabled), records validation, holiday calendar, server-derived `markedBy`, history by enrollment. Staff attendance: handled self check-in idempotent, preserves admin rows; missing validation of `records` shape and date.
- **Exams:** handled: Draft→Submitted→Approved/UnderReview workflow, admin-only verify, audit log, optional-subject enrollment, weighted results, rank. Missing: invalidation on later edits, transactions (BIZ-006).
- **Students:** handled: enrollment history, promotion log, withdrawal states, duplicate admission number. Missing: see BIZ-008.
- **Communication:** handled: school-scoped fan-out queries, batch 500/100, token pruning, Individual-audience recipient checked against schoolId. Missing: durable delivery (BIZ-009). Support tickets: school-scoped by `actor.schoolId` for real sessions (good) but see API-001; reply and status routes scope by schoolId; attachment URL unvalidated (API-007).

Positive observations

- Most mutating admin routes use `requirePermission` with a specific key; parent and teacher mobile routes consistently use `requireParent` / `requireTeacher` with ids taken from the session, never the request.
- Lib-level queries nearly always include `schoolId` (e.g. `findFirst({id, schoolId})` before update/delete on classes, subjects, calendar events, exams, notices), including `StudentFee` / `Payment` writes.
- Money is stored as integers; payment amount never comes from the client; Razorpay secrets encrypted at rest per school.
- Student promotion/update/withdrawal are transactional with an audit log; delete is blocked when billing history exists.
- Previous audit items re-verified as FIXED in the tree: `fees/student/[studentId]` and `/summary` now 401 on null actor; homework/print-marksheet no longer fall back to the toggle with an admin identity; cron routes check a secret (but see BIZ-001). Previous claim of "hardcoded SuperAdmin password" is not in my scope (AUTH).

Limitations / not assessed

- ~69 of 180 route bodies not read line by line (classified by gate only); no handler was executed except unauthenticated GET probes.
- No DB queries, no load test, no EXPLAIN; indexes and query plans not assessed (database auditor).
- Razorpay behaviour against the live gateway not tested; webhook absence is from code search.
- `app/actions/auth.js` login logic only read for the role-toggle side effect (AUTH owns it).
- Vercel `waitUntil` / `after` semantics for un-awaited promises not verified against the `node_modules/next/dist/docs` version in use.

05 - Database and Data Integrity Audit (auditor a5)

Scope

`prisma/schema.prisma` (1,775 lines, 52 models, read in full), `prisma/migrations`, `prisma/seed.js` (read only), the `lib/*.js` write paths that enforce (or fail to enforce) integrity, and read-only catalog/orphan queries against the LOCAL PostgreSQL (host confirmed `localhost:5432` by the script before connecting; credentials never printed; no EXPLAIN ANALYZE, only plain `EXPLAIN` on three queries).

Method

1. Read the whole schema; classified every relation by `onDelete`, every tenant-owned model by `schoolId`, every unique/index.
2. Throwaway Node script outside the repo (scratchpad) ran only `SELECT`/catalog queries: `pg_stat_user_tables`, `pg_indexes`, `pg_constraint`, `_prisma_migrations`, and orphan/consistency SELECTs.
3. Traced write paths in `lib/students.js`, `lib/classes.js`, `lib/fees.js`, `lib/teachers.js`, `lib/parentAccounts.js`, `lib/rbac.js`, `lib/attendance.js`.

Verified vs only read: DB facts below marked "(local DB)" were queried. Everything else is code-based, not executed. The local DB is a small demo dataset (3 schools, 190 students, 163 attendance rows), so row counts and EXPLAIN plans say nothing about scale; the planner chooses seq scans on tiny tables regardless of indexes.

Summary table

ID	Sev	Conf	Title	Component
DB-001	P1	High	Migration history does not represent the schema (1 init migration, 2 of 52 models)	Migrations / DevOps
DB-002	P1	High	Manual fee collection is a read-modify-write outside the transaction: double-collect / overpay race	Fees / Payment
DB-003	P1	High	Login identity is looked up by email only, but uniqueness is per school (or absent): ambiguous account resolution	Auth identity tables
DB-004	P2	High	Missing indexes: no `schoolId`-leading index on Payment, StudentFee, Notice, CalendarEvent; unindexed FK columns	Indexes
DB-005	P2	High	Hard delete of a Student cascades away exam results, marks, enrollment history and the "append-only" promotion log	Deletion / archival
DB-006	P2	High	All business dates are `String`; school `timezone` setting is never used to compute "today"	Date design
DB-007	P2	High	Attendance / assignments / timetable stored as JSON arrays of ids with no FK, no per-student uniqueness, last-writer-wins saves	Attendance, Teacher.assignments
DB-008	P2	Medium	Seed script is stale against the schema and embeds plaintext demo credentials and fixed `school-1` ids	prisma/seed.js
DB-009	P3	High	Status/type fields mix real enums with free strings	Schema
DB-010	P3	High	Money design: whole-rupee Int, derived payable amount, no DB invariants, hard-coded INR/paise	Fees
DB-011	P3	High	DB-level invariants documented as "enforced in code" are not enforced (one active session, one class-teacher per session)	Sessions / Sections
DB-012	P3	High	Child tables without `schoolId` (ReportLock cannot be tenant-scoped at all)	Tenant ownership

ID	Sev	Conf	Title	Component
DB-013	P4	High	Dead / stale schema elements	Schema hygiene

Findings

DB-001 - Migration history does not represent the schema

- **Category:** Database / DevOps
- **Severity:** P1 **Confidence:** High **Status:** Confirmed
- **Affected component:** `prisma/migrations`, deployment
- **Evidence:** `prisma/migrations/20260830172048_init/migration.sql:1-38` creates only the `AdminStatus` enum and the `SuperAdmin` and `SchoolAdmin` tables; `prisma/schema.prisma` defines 52 models. `git log -- prisma/migrations` shows only the initial commit. `_prisma_migrations` in the local DB has exactly one row. `package.json:7` build script is `prisma generate && next build` (no `prisma migrate deploy`). The schema header comments and the existence of RBAC/support/enrollment tables in the local DB prove the schema was applied some other way (`db push` style).
- **Impact:** A new environment (staging, DR restore target, a second region) cannot be created from the repo; `prisma migrate deploy` would produce a database missing 50 tables. No reviewable, reversible record of schema change; no safe path for additive changes with data (e.g. converting date strings to DateTime). Production schema drift cannot be detected.
- **Reproduction / verification:** code-based plus local catalog query of `_prisma_migrations`; not executed against any other DB.
- **Root cause:** Schema evolved with `db push`; migrations were never re-baselined.
- **Recommended fix:** Baseline: generate a full-schema migration from the production-equivalent schema (`prisma migrate diff --from-empty --to-schema-datamodel prisma/schema.prisma --script`), mark it applied on existing databases with `prisma migrate resolve --applied`, and from then on use `migrate dev` / `migrate deploy` in the release process.
- **Validation:** Create an empty scratch database, run `migrate deploy`, then `migrate diff` against the real schema returns empty.
- **Effort:** M **Dependencies:** DB-004, DB-006 (both need migrations)

DB-002 - Manual fee collection race: stale read outside the transaction

- **Category:** Database / Backend
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code-based; race not executed)
- **Affected component:** Fees > collect payment, Razorpay verify
- **Evidence:** `lib/fees.js:1025-1058` (`collectManualPayment`): reads the fee with `getStudentFeeById`, checks `status === 'PAID'` and `remainingDue` in JS, computes `newPaidAmount = fee.paidAmount + collectAmount`, then runs `$transaction([payment.create, studentFee.update({ paidAmount: newPaidAmount })])`. The transaction does not re-read or lock the fee, and `paidAmount` is written as an absolute value. `lib/fees.js:1194-1250` (`verifyRazorpayPayment`) has the same shape (`fee.paidAmount + payment.amount`). There is no unique key or CHECK constraint tying `sum(Payment.amount where SUCCESS)` to `StudentFee.paidAmount` or capping it at the payable amount (schema `StudentFee`, `Payment`, lines 537-557 and 1427-1443).
- **Impact:** Two accountants (or a double click, or a manual collect racing a pending Razorpay verify for the same fee) both pass the "not PAID" check; two SUCCESS Payment rows are created while `paidAmount` reflects only one (or a Razorpay verify adds on top of a manual full payment). Ledger and fee row then disagree; the school's reconciliation breaks and parents can be double-charged. Local DB: fee rows are currently consistent (0 overpaid), so this is a latent defect, not observed corruption.

- **Reproduction / verification:** code-based. Safe test: in a scratch DB, fire two concurrent `collectManualPayment` calls for the same `studentFeeId`.
- **Root cause:** Check-then-act with application-computed absolute totals and no row lock/optimistic condition.
- **Recommended fix:** Inside one interactive transaction do the guarded update: `updateMany({ where: { id, schoolId, paidAmount: fee.paidAmount }, data: { paidAmount: { increment: amount }, status } })` and abort if `count === 0`; or `SELECT ... FOR UPDATE` via `$queryRaw`. For Razorpay mark the pending Payment SUCCESS with `updateMany({ where: { id, status: 'PENDING' } })` first and only continue if it matched.
- **Validation:** Concurrency test (two parallel calls) yields exactly one SUCCESS Payment and `paidAmount` equals the sum of SUCCESS payments.
- **Effort:** S-M **Dependencies:** REL-001

DB-003 - Login identity resolved by email only; uniqueness is per school or absent

- **Category:** Database / Security
- **Severity:** P1 **Confidence:** High **Status:** Confirmed
- **Affected component:** Teacher / Parent / Principal-Accountant / SchoolAdmin sign-in
- **Evidence:** Schema: `Teacher` has no unique on `loginEmail` or `email` (only `@@unique([schoolId, employeeId])`, `schema.prisma:678`); `ParentAccount` is `@@unique([schoolId, email])` (285); `User` is `@@unique([schoolId, email])` (153); `SchoolAdmin.email` is globally unique (322-326). Lookups: `lib/teachers.js:436-440` and `:460-463` `findFirst({ where: { loginEmail: ... } })` with no school; `lib/parentAccounts.js:108-111` `findFirst` by email only; `lib/rbac.js:44-49` `user.findFirst` by email for Principal/Accountant. Local DB: one group of 4 teachers in one school shares the same `loginEmail` (demo data), 13 of 26 teachers have login enabled.
- **Impact:** Two schools can legitimately register the same parent or teacher email (the schema permits it), after which `findFirst` returns an arbitrary row: the person logs into the wrong tenant, or a correct password for school B fails because school A's row was returned first (a person who is a parent at two schools cannot sign in to one). Within a school, duplicate teacher login emails make password-reset and login nondeterministic. This is both an integrity and a tenant-isolation risk (credential of one tenant matched against another tenant's account when passwords coincide).
- **Reproduction / verification:** code-based; local duplicate group confirmed by `GROUP BY lower("loginEmail") HAVING count(*) > 1`.
- **Root cause:** Tenant-scoped uniqueness in the schema, global lookup in the login flows; no uniqueness at all for Teacher emails.
- **Recommended fix:** Decide the product rule. Simplest consistent option: login emails are globally unique across tenants (unique index on lowercased email per identity table, or a single `User` identity table), with a data-cleanup migration first. Alternative: require a school code at login and look up `{ schoolId, email }`. Add `@@unique([schoolId, loginEmail])` to Teacher at minimum.
- **Validation:** Unique violations on duplicate create; login test with the same email in two schools.
- **Effort:** M **Dependencies:** REL-003, DB-001

DB-004 - Index gaps against the hot tenant-scoped query patterns

- **Category:** Database / Performance
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (index set); impact at scale not measured
- **Affected component:** Payments, student fees, notices, calendar, FK lookups
- **Evidence:** `pg_indexes` (local DB) and schema. Query patterns from `lib/*.js` vs index:
  - Students list/roster by `(schoolId, academicSession, class, section, status)`: `Students_schoolId_academicSession_class_section_status_idx` present. Good. Searches (`contains`, insensitive, on names/admissionId, `guardian` JSON path, `lib/students.js:268-286`) cannot use it beyond `schoolId`.
  - Attendance by date/class: unique `(schoolId, academicSession, date, className, sectionName)` serves day and date-range lookups. Good. Dashboard trend `WHERE schoolId` scans all rows of the school (see PERF-

003).
  - Payments by school/status/date (`lib/fees.js:704-720`, `1408-1416`, `1537`, `1551`, `lib/dashboard.js:342`): the only indexes on `Payment` are the two `razorpay*` uniques. No `(schoolId, status, paidAt)` / `(schoolId, createdAt)` index; `Payment.studentId` and `Payment.studentFeeId` (FKs, used by ledger and fee detail) are unindexed.
  - StudentFee by school (`loadSchoolFees`, `lib/fees.js:1340`; groupBy at `:1413`): only `(studentId, academicSession, term)` unique exists; no `schoolId`-leading index. Local `EXPLAIN` of the payments query uses a Seq Scan (tiny table, not conclusive).
  - Notices (`lib/notices.js:58`), CalendarEvents (`lib/calendarEvents.js:40`), Subjects/Classes lists: filtered on `schoolId` with `orderBy createdAt/startDate`; Notice and CalendarEvent have no index at all.
  - Per-student lookups with no student-leading index: `ParentStudentLink(studentId)` (used by notification fan-out and fees, `lib/notices.js:477`, `lib/fees.js:996`; only `(parentAccountId, studentId)` unique exists), `ExamMark(studentId)`, `ExamResult(studentId)` (`lib/examResults.js:245`), `StudentEnrollments(classId)`, `(sectionId)`, `Sections(classTeacherId)`, `TeacherLeave(teacherId)`, `SubjectTests(teacherId, subjectId)`.
  - Postgres does not index FK columns automatically, so the cascades in DB-005 (delete Student) and Class/Section/Teacher deletes also seq-scan the child tables.
- **Impact:** Fine at demo size. At the stated target (hundreds of thousands of payments, millions of ExamMarks, ParentNotifications growing with every notice x student) these become sequential scans shared across all tenants, plus slow deletes. Not a proven bottleneck; marked P2 because the fix is cheap and the table with highest write volume (Payment, ParentNotifications) lacks school-leading access paths.
- **Reproduction / verification:** index listing from `pg_indexes`; plans on a 17-row table are not evidence. Validate with `EXPLAIN (ANALYZE)` on a seeded 100k-student copy (not on this DB).
- **Root cause:** Indexes were added reactively for roster/exam tables; fees/payments/notices modules were not revisited.
- **Recommended fix:** `@@index([schoolId, status, paidAt])` and `@@index([studentId])`, `@@index([studentFeeId])` on Payment; `@@index([schoolId, academicSession])` on StudentFee; `@@index([schoolId, createdAt])` on Notice; `@@index([schoolId, startDate])` on CalendarEvent; `@@index([studentId])` on ParentStudentLink, ExamMark, ExamResult; `@@index([classId])` / `[sectionId]` on StudentEnrollment. Consider `pg_trgm` GIN indexes if student search becomes slow.
- **Validation:** Plans on a scaled dataset show Index Scan; write-throughput regression check on Payment inserts.
- **Effort:** S **Dependencies:** DB-001 (ship as migration)

DB-005 - Student hard delete destroys academic history and the audit trail

- **Category:** Database
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** Students > delete
- **Evidence:** `lib/students.js:673-698` `deleteStudent` runs `prisma.student.delete`; it only fails (P2003) if StudentFee/Payment/MonthlyReport/YearlyReport/SubjectTestMark rows exist (Restrict). `pg_constraint` (local DB) and schema show Cascade from Student to `ExamMark`, `ExamResult`, `ExamOptionalEnrollment`, `StudentEnrollment`, `StudentPromotionLog`, `ParentNotification`, `ParentStudentLink`. `schema.prisma:1036-1040` documents `StudentPromotionLog` as "append-only ... never updated/deleted". The student's id also stays inside `Attendance.records` JSON.
- **Impact:** A student who has exam marks and results but no fee row (e.g. fees not generated, or a school not using the fee module) can be hard-deleted, silently deleting published results, enrollment history and the promotion log (data a school is typically required to retain). Which students are deletable depends on an unrelated module's data. No soft-delete column exists; `status` TransferredOut/Graduated is the workaround and is advisory only. All 20+ `School` relations are Restrict, so a school cannot be offboarded/purged by a single delete and there is no documented tenant export/purge procedure.
- **Reproduction / verification:** code-based; not executed.

- **Root cause:** Delete policy was derived from "what happens to block it" rather than a retention rule.
- **Recommended fix:** Make delete = status change (or an `archivedAt DateTime?` column) for any student with enrollments/results; restrict hard delete to students with no academic or financial history; change `StudentPromotionLog.student` and `ExamResult.student` to Restrict.
- **Validation:** Attempt delete on a student with marks only: refused.
- **Effort:** M **Dependencies:** REL-002

DB-006 - Business dates stored as `String`; school timezone is not used

- **Category:** Database
- **Severity:** P2 **Confidence:** High (storage) / Medium (UTC server-clock effect, Vercel TZ not inspected) **Status:** Confirmed
- **Affected component:** Admission, attendance, exams, fee term dates, leave, calendar, notices, homework
- **Evidence:** `String` date columns: `Student.admissionDate`, `dob`; `Teacher.dob`, `joiningDate`, `relievingDate`; `Attendance.date`; `TeacherAttendance.date`; `Exam.startDate/endDate`; `ExamSchedule.examDate`; `FeeStructureTerm.startDate`; `AcademicSession.startDate/endDate`; `CalendarEvent.startDate/endDate`; `Notice.expiryDate`; `Homework.assignedDate`; `TeacherLeave.startDate/endDate`; `MonthlyReport.month`. Month filters use string prefixes (`lib/dashboard.js:101,109` `startsWith`), ranges rely on lexicographic order (`lib/attendance.js:174` `gte/lte` on strings). Local DB: all 163 attendance dates and 190 admission dates are well-formed 10-char values (no malformed data observed today). `lib/dateUtils.js:1-25` computes "today" from the server's local clock (`getFullYear/getMonth/getDate`), `lib/exams.js:128`, `lib/examMarks.js:413` use `toISOString().slice(0,10)` (UTC). `School.timezone` is saved (`lib/schoolSettings.js:151`) but grep finds no use of it in any date computation.
- **Impact:** No DB-level validation (any string can be stored); correctness depends on every writer emitting `YYYY-MM-DD`. On Vercel the server clock is normally UTC, so for an Asia/Kolkata school between 00:00 and 05:30 local time "today" is the previous day: attendance "mark today", the 14:00 deadline (`School.attendanceDeadlineTime`, applied with server-local `setHours`), exam "upcoming/ongoing" and fee-term generation can act on the wrong day. Date arithmetic and reporting in SQL require casts.
- **Reproduction / verification:** code-based. Check Vercel function `TZ` and compare behaviour at 23:00 UTC.
- **Root cause:** Dates modelled as display strings; per-school timezone captured but not applied.
- **Recommended fix:** Smallest effective step: one `schoolToday(timezone)` helper (using `Intl.DateTimeFormat('en-CA', { timeZone })`) used by every "today" computation. Longer term, migrate date columns to `@db.Date` with a CHECK or a backfill migration (needs DB-001 first).
- **Validation:** Unit check of helper at 20:00 UTC for Asia/Kolkata returns the next calendar day.
- **Effort:** S (helper) / L (column migration) **Dependencies:** DB-001

DB-007 - Relationships held in JSON arrays: no FK, no uniqueness, whole-array overwrite

- **Category:** Database
- **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Affected component:** Attendance, TeacherAttendance, Teacher.assignments, TimeTable.schedule, Student JSON blobs
- **Evidence:** `Attendance.records` (`schema.prisma:874`), `TeacherAttendance.records` (723), `Teacher.assignments` (661), `TimeTable.schedule` (812), `Student.guardian/feeDetails/documents` (387-400), `CalendarEvent.appliesToClasses/Sections`, `GeneratedReportJob.targetIds String[]`. `lib/attendance.js:57-69` `saveAttendance` upserts the whole `records` array supplied by the client. Local DB orphan checks (all 0): attendance record ids not in Students; teacher assignments referencing a non-existent Class; notices pointing at missing teachers. 163 attendance rows hold 1,700 student marks (about 10 per row) in the demo data.
- **Impact:** (a) Student/teacher ids inside JSON are never checked by the DB: deleting or transferring a student leaves dangling ids and a student can appear twice in one `records` array. (b) Concurrency: two users saving the same class+date (teacher plus admin override) overwrite each other (last writer wins, no version/`updatedAt`

precondition). (c) Per-student attendance queries cannot use an index; every report scans and `find()`s inside arrays (PERF-009). (d) The design is a conscious trade-off (documented at `schema.prisma:852-862`), reasonable for write amplification, but it makes attendance the one core record without referential integrity. `Student.feeDetails` JSON duplicates fee concepts that also live in StudentFee (see REL-005).

- **Reproduction / verification:** code-based plus local orphan SELECTs.
- **Root cause:** Denormalisation chosen for write cost and speed of development.
- **Recommended fix:** Keep the row-per-section shape but (1) add an `updatedAt`/version precondition to `saveAttendance`, (2) validate that every `studentId` belongs to the section's enrolled roster before saving (one query), (3) when a student is removed, either block or purge ids from records. A row-per-student `AttendanceEntry(attendanceId, studentId, status)` table is the proper fix if per-student reporting becomes a hot path; benchmark first.
- **Validation:** Duplicate/foreign student id in `records` rejected; concurrent save returns a conflict.
- **Effort:** M **Dependencies:** PERF-003, PERF-009

DB-008 - Seed script out of date with the schema; demo credentials and fixed ids

- **Category:** Database / DevOps
- **Severity:** P2 **Confidence:** Medium **Status:** Confirmed (code-based; script not run per rules)
- **Affected component:** `prisma/seed.js`, `package.json` `db:seed`
- **Evidence:** `prisma/seed.js:148-152` updates `Student.portalPasswordHash`, a field that no longer exists in the schema (parent logins moved to `ParentAccount`, `schema.prisma:401-403`); `prisma/seed.js:112-113` seeds two school-admin accounts with a shared plaintext demo password literal; `:148` a parent demo password literal; `:3-8,84-92` the SuperAdmin password comes from an env var or is generated and printed to stdout. Fixed ids `school-1`/`school-2` mirror `lib/school.js:5` `SEED_SCHOOL_ID`, which production code still uses as a fallback tenant (see PERF-001).
- **Impact:** `npm run db:seed` will fail against the current schema (Prisma rejects an unknown field) so the documented bootstrap path for a new environment is broken. If it is patched and run in production, known demo credentials for admin and parent accounts would be created. Type only reported here; values are not repeated.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Seed not maintained when the parent-account model replaced per-student portal passwords.
- **Recommended fix:** Split into `seed-reference.js` (roles, permissions, grade scale: safe for prod) and `seed-demo.js` (guarded by `NODE_ENV !== 'production'`); remove stale fields; generate random demo passwords.
- **Validation:** Fresh DB: `migrate deploy` then reference seed succeeds; demo seed refuses to run in production.
- **Effort:** S **Dependencies:** DB-001

DB-009 - Status/type columns: enums mixed with free strings

- **Category:** Database
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** Enums exist for StudentStatus, FeeTerm, PaymentStatus, ExamStatus, ExamMarkStatus, etc. Free `String` for: `AcademicSession.status` (`schema.prisma:986`, "Active/Upcoming/..."), `ExamResult.status` (1266), `ExamMark.attendanceStatus` (1226), `Notice.audience/priority/recipientType` (925-938), `CalendarEvent.category`, `PushToken.ownerRole/platform`, `StudentPromotionLog.action`, `ExamAuditLog.action`, `Exam.examType` (intentionally open), `Attendance` record `status` inside JSON, `ReportLock.userId`.
- **Impact:** A typo in code writes a status no query matches; no DB protection. Low today because values come from shared constant arrays; risk grows with more writers (mobile, API).
- **Recommended fix:** Convert the closed sets (AcademicSession.status, ExamResult.status, Notice.audience) to enums when next touching them; add CHECK constraints where enums are unwanted.
- **Validation:** Insert of an unknown value fails.
- **Effort:** S-M **Dependencies:** DB-001

DB-010 - Money columns: Int rupees, derived payable amount, no invariants

- **Category:** Database
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** All amounts are `Int` (`StudentFee.totalAmount/paidAmount`, `Payment.amount`, discounts, late fees); no Float/Decimal for money. `lib/fees.js:100-104` rounds percent discounts with `Math.round`; late fees accrue per day at read time (`discountAmountFor`, `lateFeeAmountFor`, `daysOverdueFor`), so `payableAmount` is never stored and changes as time passes. `lib/fees.js:1157,1166` hard-code `* 100` paise and `currency: 'INR'` while `School.currency` exists. No CHECK that `paidAmount <= payable` or `amount > 0`; no receipt number or refund/reversal row type (`PaymentStatus` has no REFUNDED); abandoned Razorpay orders stay PENDING forever (local DB: 9 PENDING vs 4 SUCCESS Razorpay payments).
- **Impact:** Whole rupees are acceptable for Indian school fees but cannot represent paise-level concessions; the stored `status` (PAID) can become inconsistent with a later-changing computed payable amount; no audit-grade receipt sequence per school. Not a correctness bug today.
- **Recommended fix:** Add CHECK constraints (`paidAmount >= 0`, `amount > 0`); add a per-school receipt number on SUCCESS payments; freeze late fee into the StudentFee (or record it as a StudentFeeItem) when applied; add a PENDING-order expiry job. Keep Int but document the unit as rupees.
- **Validation:** Constraint violations rejected; reconciliation query `sum(Payment) = paidAmount` returns zero mismatches.
- **Effort:** M **Dependencies:** DB-002, REL-001

DB-011 - Invariants documented as "enforced in code" are not enforced

- **Category:** Database
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** `schema.prisma:826-829` (one class teacher per section per session), `:976-977` (exactly one Active AcademicSession per school), `User` nullable `schoolId` unique (`:147-153`, acknowledged). Local DB checks: 0 violations of either invariant today.
- **Impact:** Concurrent requests or direct DB edits can violate them. Postgres partial unique indexes handle both: `CREATE UNIQUE INDEX ... ON "AcademicSessions"("schoolId") WHERE status = 'Active'`; `ON "Sections" ("classTeacherId","academicSession") WHERE "classTeacherId" IS NOT NULL`; `ON "User"(lower(email)) WHERE "schoolId" IS NULL`. Prisma cannot declare them but they can live in a migration (needs DB-001).
- **Effort:** S **Dependencies:** DB-001

DB-012 - Tenant ownership gaps

- **Category:** Database
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** Models without a `schoolId` column: `StudentFeeItem`, `FeeStructureTerm`, `FeeStructureItem`, `SubjectTestMark`, `MonthlyReportActivity/Holistic/Remark`, `SupportTicketReply` (all reachable only through a tenant-owned parent, acceptable), plus `ReportLock` (`schema.prisma:1648-1658`: polymorphic `reportId`, `userId` with no FK and no `schoolId`, so a tenant filter on it is impossible) and `SubjectTestMark.studentId` which is not tied to the test's school by any constraint. Global-by-design: `Role`, `Permission`, `RolePermission`, `SuperAdmin`. Per-school-scoped uniqueness exists for Student.admissionId, Teacher.employeeId, Class, Section (via class), Subject, Session, ParentAccount, User (nullable). `ParentStudentLink.schoolId` is denormalised but nothing enforces that it equals both sides' school (comment at `schema.prisma:288-294` admits it is app-checked).
- **Impact:** Cross-tenant joins are blocked only by application code. ReportLock rows cannot be listed or purged per tenant.
- **Recommended fix:** Add `schoolId` + index to `ReportLock`; for the ParentStudentLink invariant use composite FKs `(schoolId, studentId)` -> `Student(schoolId, id)` (requires `@@unique([schoolId, id])` on Student and ParentAccount).

- **Effort:** S-M **Dependencies:** DB-001

DB-013 - Dead or stale schema elements

- **Category:** Database
- **Severity:** P4 **Confidence:** High **Status:** Confirmed
- **Evidence:** No code references `GeneratedReportJob` (grep over `lib`, `app`: none; 0 local rows), `MonthlyReport.pdfUrl`, `YearlyReport.pdfUrl`; `School.studentCount/teacherCount` are informational ints replaced by live counts (`lib/schools.js:42-64`); `SchoolAdmin.permissions Json` and the header comments in `schema.prisma:23-66` describe a pre-RBAC cut-over that is partly done (see REL-003). `FeeStructureItem.required` and late-fee columns were "captured for future" per the schema comment at `:508-511` (late fee is now used, `required` not).
- **Impact:** Misleading model surface; maintenance cost.
- **Recommended fix:** Remove or mark as planned; refresh header comments.
- **Effort:** S

Data-integrity checks run on the local DB (all read-only)

Check	Result
Students without a matching Class row (school, name, session)	0
Students without a StudentEnrollment	0
Students whose section differs from their current-session enrollment	0
StudentFee without items / `totalAmount` != sum(items)	0 / 0
StudentFee `paidAmount` != sum(SUCCESS payments)	0
StudentFee `paidAmount` > `totalAmount`	0
Payment whose school differs from its student's school	0
Attendance record ids not in Students	0
Teachers sharing a `loginEmail`	1 group of 4
Teachers with no linked `User` row	17 of 26 (also 1 ParentAccount, 1 SchoolAdmin)
Two class-teacher assignments per session / two Active sessions per school	0 / 0
PENDING Razorpay payments	9 (vs 4 SUCCESS)
Dates not matching the 10-char ISO shape	0 (attendance, admission)

FK / onDelete summary (from `pg_constraint`, local DB)

- Cascade: Section->Class; ExamSchedule/ExamMark/ExamResult/ExamAuditLog->Exam; ExamMark->ExamSchedule; Student-> ExamMark, ExamResult, ExamOptionalEnrollment, StudentEnrollment, StudentPromotionLog, ParentNotification, ParentStudentLink; ParentStudentLink->ParentAccount; fee structure terms/items; StudentFeeItem->StudentFee; report children->MonthlyReport; SubjectTestMark->SubjectTest; TeacherNotification->Teacher; SupportTicketReply->Ticket; RolePermission.
- Restrict: every `School` FK; StudentFee/Payment/MonthlyReport/YearlyReport/SubjectTestMark -> Student; Payment->StudentFee; StudentEnrollment->Class/AcademicSession; SubjectTest->Teacher/Subject; TeacherLeave->Teacher; User->Role.
- SetNull: Section.classTeacherId, StudentEnrollment.sectionId, the `userId` links on SuperAdmin/SchoolAdmin/Teacher/ParentAccount, User.schoolId, Exam.parentExamId.
- Plain strings (no FK, historical attribution): Notice.postedByTeacherId, Homework.assignedByTeacherId, ExamMark.enteredByTeacherId/verifiedByAdminId, ExamAuditLog.actorId, SupportTicket.raisedByUserId, PushToken.ownerId, MonthlyReport.lockedByUserId, ParentNotification.sourceId.

Positive observations

- Tenant-scoped composite uniques for admission id, employee id, class, subject, session, attendance, fee structure and timetable; every tenant-owned business table carries an indexed `schoolId` or hangs off one.
- Razorpay ids are unique (`schema.prisma:1438-1439`) and the verify path handles duplicate delivery.
- Fee generation is idempotent by `@@unique([studentId, academicSession, term])` and snapshots items so structure edits never rewrite paid fees.
- `StudentEnrollment` plus `StudentPromotionLog` now capture session/class/section history, and promotion runs in one transaction (`lib/students.js:744-777`); `ExamResult` freezes class/section at generation.
- Student delete is blocked when billing history exists; Class and level-rename are blocked while students reference them (`lib/classes.js:289-367`).
- Local data is internally consistent on every orphan check above.

Limitations / not assessed

- Production database (Supabase/Sydney) not touched per rules; indexes, row counts, bloat and connection settings there were not inspected. Local DB has 3 schools and ~190 students, so no scale conclusions come from it.
- No EXPLAIN ANALYZE, no load test; index recommendations are pattern-based.
- Backup, PITR, retention and restore procedures: not assessable from the repo.
- Column-level encryption of Aadhaar numbers/bank details (stored as plain columns such as `Student.aadhaarNumber` and in the `Teacher.bankDetails` JSON) is a privacy topic owned by another auditor.

08 — Mobile Application Audit (Expo / React Native, `schoolapp360`)

Auditor: A2. Scope: `C:\Users\Tashaf\Desktop\SchoolMS\schoolapp360` plus the backend routes it calls (`ADMIN/app/api/auth/*`, `notifications/register-token`, `lib/auth/jwt.js`, `lib/pushTokens.js`). Code-based; no build, install, test or device run. No sensitive file was opened.

Method

- Listed the repo (104 files under `src/`, about 24.7k lines) and read in full: `src/api/baseApi.js`, `core/secureAuthStorage.js`, `core/store.js`, `core/sentry.js`, `App.js`, `core/navigation/RootNavigator.js`, `features/auth/{authSlice,authApi}.js`, `features/notifications/*`, `OfflineBanner.js` (top), relevant parts of `LoginScreen.js`, `ProfileScreen.js`, `ParentFeesScreen.js`, `feesReceipt.js`, `app.json`, `package.json`, `jest.config.js`, `jest.setup.js`, `.gitignore`, `.env.example` (names only). Greps over `src/` for logging, storage, linking, accessibility, tests.
- Docs (`README.md`, `ARCHITECTURE.md` etc.) were not reviewed in detail.
- Cross-checked backend: teacher login route (`app/api/auth/teacher/login/route.js`), JWT lifetime (`lib/auth/jwt.js:10`), `session.js` Bearer fallback (`:30-36`), push-token route and `lib/pushTokens.js`.

Sensitive files (existence and git status only; contents NOT read)

File	Exists	Git status	Holds secret material
`google-services.json`	yes	untracked, NOT ignored	Firebase client config (not a private key, but project identifiers)
`eas.json`	yes	untracked, NOT ignored	build/env config; treat as sensitive per audit rules
`*-firebase-adminsdk-*.json`	yes	ignored (`.gitignore:42`)	yes — Firebase service-account private key
`keystore-backup/` (contains a release keystore)	yes	ignored (`/keystore-backup`)	yes — Android signing key
`android/` (generated, incl. `sentry.properties`)	yes	ignored (`/android`)	may contain Sentry auth material (not opened)
`.env.local`	yes	ignored	env values (not opened)

Summary table

ID	Sev	Conf	Title	Component
MOB-001	P1	High	Demo teacher/parent credentials hard-coded and shown in the production login screen	LoginScreen
MOB-002	P2	High	Real app is not under version control (only Expo template committed, no remote); config files untracked	repo / DevOps
MOB-003	P2	High	No 401/expired-token handling; logout does not clear RTK Query cache	baseApi, ProfileScreen
MOB-004	P2	High	Push token never unregistered on logout	usePushNotifications, backend
MOB-005	P2	Medium	Razorpay verify failure reported as "Payment failed" after money captured	ParentFeesScreen
MOB-006	P3	High	Silent `http://localhost:3000/api` fallback; login error reveals API URL	baseApi, LoginScreen

ID	Sev	Conf	Title	Component
MOB-007	P3	High	Zero tests despite jest setup	testing
MOB-008	P3	High	No accessibility labels (0 `accessibilityLabel` vs 670 `TouchableOpacity`)	UI
MOB-009	P3	Medium	Offline handling is banner-only; no write queue for attendance/marks	offline
MOB-010	P4	High	Role/screen selection is client-side (AsyncStorage); backend enforces from JWT	auth
MOB-011	P4	Low	Generated manifest requests `RECORD_AUDIO` / `SYSTEM_ALERT_WINDOW`	android config
MOB-012	P3	Medium	Very large screen files (up to 1,061 lines), duplicated per-role patterns	maintainability

Findings

MOB-001 — Demo credentials hard-coded and displayed in the production login screen

- **Category:** Mobile / Security **Severity:** P1 **Confidence:** High **Status:** Confirmed (bundle contents); exploitability depends on those accounts existing in the production DB
- **Affected component:** `src/features/auth/LoginScreen.js`
- **Evidence:** `LoginScreen.js:48-58` defines `TEST_CREDENTIALS` with a plaintext email+password literal for a seeded demo teacher and demo parent (values not reproduced); `:75` selects them; `:117-118` pre-fills the form; `:263` renders the "test credentials" panel with email and password in plain text. Nothing is gated by `__DEV__`.
- **Impact:** Every installed production build ships and displays working-looking credentials for named seed accounts. If those seed accounts exist in the production database (the web seed script creates them), anyone with the APK gets authenticated access to a school's teacher/parent data.
- **Reproduction / verification:** code-based; not executed. Whether production contains these accounts was not checked (no production access).
- **Root cause:** Developer convenience block left in release code.
- **Recommended fix:** Wrap in `__DEV__` (`const TEST_CREDENTIALS = __DEV__ ? {...} : {}`), delete the accounts from production, rotate their passwords.
- **Validation:** Release bundle contains no such strings; panel absent in a release build.
- **Effort:** S **Dependencies:** none.

MOB-002 — Real app is not under version control; config files untracked

- **Category:** DevOps / Mobile **Severity:** P2 **Confidence:** High **Status:** Confirmed
- **Affected component:** `schoolapp360` git repo
- **Evidence:** `git -C schoolapp360 log` has one commit ("Initial commit"), no remote; `git ls-files` lists only the Expo starter files (55) while `src/App.js`, all of `src/features`, `eas.json`, `google-services.json`, `plugins/` are untracked and the starter `src/app/*` files show as deleted. `.gitignore` comment says `google-services.json` "stays tracked" but it is untracked.
- **Impact:** Entire mobile source exists only on one machine (no backup/history/review); a careless `git add .` would stage `eas.json` and `google-services.json` (the service-account key and keystore are correctly ignored).
- **Reproduction / verification:** `git status`, `git ls-files`, `git remote -v`, `git check-ignore` (read-only).
- **Root cause:** Project scaffolded from the Expo template and never committed.
- **Recommended fix:** Commit the app (after deciding whether `eas.json` belongs in git — it normally does when it holds no secrets), add a private remote. Keep the adminsdk key and keystore out of the repo and back them up in a secret store.

- **Validation:** `git ls-files` shows `src/`; a fresh clone builds.
- **Effort:** S **Dependencies:** none.

MOB-003 — No 401/expired-token handling; logout leaves RTK Query cache

- **Category:** Mobile / Authentication **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `src/api/baseApi.js`, `src/features/profile/ProfileScreen.js`, backend JWT
- **Evidence:** `baseApi.js` only attaches `Authorization` (`:6-14`) and retries GET on transient errors (`:25-37`); no branch handles `status === 401`. `grep logout|401|removeAuthToken` shows the only logout path is `ProfileScreen.js:62-70`, which clears the token, three AsyncStorage keys (including a `refresh_token` key that nothing ever sets) and dispatches `logout()`; there is no `baseApi.util.resetApiState()` (grep returns none). Server token lifetime is 7 days with no refresh (`ADMIN/lib/auth/jwt.js:10`).
- **Impact:** When the token expires or is invalid, every screen shows a generic load error and the user stays "logged in" until they manually log out. After a manual logout/login by a different user on the same device, cached queries (RTK `refetchOnMountOrArgChange: 30`, `baseApi.js:79`) from the previous account can be shown briefly until refetch.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Minimal baseQuery; logout does not reset API state.
- **Recommended fix:** In `baseQuery`, if `result.error?.status === 401` then clear secure token/AsyncStorage and `api.dispatch(logout())`; in logout also `dispatch(baseApi.util.resetApiState())`.
- **Validation:** Use an expired token: app returns to login once; no stale data for a second user.
- **Effort:** S **Dependencies:** MOB-004.

MOB-004 — Push token is never unregistered on logout

- **Category:** Mobile / Privacy **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `src/features/notifications/*`, `ADMIN/lib/pushTokens.js`, `app/api/notifications/register-token`
- **Evidence:** the app has a register mutation only (`pushTokenApi.js`); the backend has a single route `register-token` (no delete/unregister route; `deletePushToken` in `lib/pushTokens.js:46` is used for dead tokens). `registerPushToken` upserts by token (`lib/pushTokens.js:8-9`), so the token stays owned by the previous user until someone else signs in on that device.
- **Impact:** After logout (shared family/teacher device) notifications for the previous account (fee, exam result, notice content) continue to arrive on the device.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** No unregister endpoint/call.
- **Recommended fix:** Add `DELETE /api/notifications/register-token` (by token, caller must own it) and call it before clearing the auth token in `confirmLogout`.
- **Validation:** After logout, a test push to the old user is not delivered to that device.
- **Effort:** S **Dependencies:** MOB-003.

MOB-005 — Razorpay verify failure shown as "Payment failed" after capture

- **Category:** Mobile / Financial **Severity:** P2 **Confidence:** Medium **Status:** Confirmed (code-based)
- **Affected component:** `src/features/parent/ParentFeesScreen.js`
- **Evidence:** `:149-190` — checkout open then `verifyRazorpayPayment(...).unwrap()` inside one `try`; any error other than checkout code 0/2 shows `Alert 'Payment failed'`. The verify mutation is a POST and is (correctly) not auto-retried (`baseApi.js:30-36`). Pay buttons are disabled via `payingTermId` state (`:263,:378`), which is set before the async work.
- **Impact:** Network loss after a successful charge produces "Payment failed"; user may pay again. Combined with the missing webhook (FE-001) the fee is not recorded.

- **Reproduction / verification:** code-based; not executed.
- **Root cause:** No distinction between checkout failure and verify failure.
- **Recommended fix:** Track the checkout success; on verify failure show "Payment received, confirming with the school" and refetch Fees; fix server-side via webhook (FE-001).
- **Validation:** Force the verify call to fail after a test payment; message and eventual recording are correct.
- **Effort:** S **Dependencies:** FE-001.

MOB-006 — Silent localhost fallback; login error discloses API URL

- **Category:** Mobile / Configuration **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Affected component:** `src/api/baseApi.js:3`, `LoginScreen.js:~106-112`
- **Evidence:** `API_BASE_URL = process.env.EXPO_PUBLIC_API_URL || 'http://localhost:3000/api'`; on `FETCH_ERROR` the login screen tells the user to "check that the backend is running" and prints the configured URL. `eas.json` defines `EXPO_PUBLIC_API_URL` per build profile (names only inspected via key listing, values not read). Only four `EXPO_PUBLIC_*` vars are used: API URL, Sentry DSN, crash-reporting flag, Razorpay key id (all public by design).
- **Impact:** A release build missing the variable silently targets localhost (all requests fail); developer-oriented error text shown to end users.
- **Reproduction / verification:** code-based.
- **Root cause:** Dev defaults in production code path.
- **Recommended fix:** Fail loudly if unset outside `__DEV__`; show generic user message.
- **Validation:** Release build without the var surfaces a clear config error in logs.
- **Effort:** S **Dependencies:** none.

MOB-007 — No tests

- **Category:** Mobile / Testing **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Evidence:** `jest.config.js` and `jest.setup.js` exist; `find src -name '*.test.*'` and `__tests__` return nothing. `npm test` not run (no network/build allowed). Lint script targets `src --ext .js,.jsx`.
- **Impact:** Auth, payment and attendance flows have no regression protection.
- **Recommended fix:** Start with baseQuery 401 handling and the fee/receipt helpers (`feesFormat.js`, `feesReceipt.js`).
- **Reproduction / verification:** file search only; tests not run.
- **Root cause:** Test scaffolding added but no tests written.
- **Validation:** `npm test` runs real tests locally.
- **Effort:** M **Dependencies:** none.

MOB-008 — No accessibility labels

- **Category:** Mobile / UX **Severity:** P3 **Confidence:** High **Status:** Confirmed (grep-based, not device tested)
- **Evidence:** `grep -rn accessibilityLabel src` returns 0; 670 `TouchableOpacity` uses.
- **Impact:** Icon-only buttons are unusable with TalkBack/VoiceOver.
- **Recommended fix:** Add `accessibilityRole` / `accessibilityLabel` to shared `Button`, header back/close and tab icons first.
- **Effort:** M **Dependencies:** none.

MOB-009 — Offline behaviour is banner-only

- **Category:** Mobile / Reliability **Severity:** P3 **Confidence:** Medium **Status:** Confirmed (code-based)
- **Evidence:** `OfflineBanner.js` (NetInfo listener, `isConnected` / `isInternetReachable`) is the only offline feature; no persisted RTK cache or write queue (no redux-persist in `package.json`). GETs retry twice on FETCH_ERROR/TIMEOUT/5xx; mutations never retry (`baseApi.js:25-37`, deliberate and good).

- **Impact:** A teacher marking attendance or entering marks on poor connectivity gets an error and must re-enter; no draft is saved.
- **Recommended fix:** Keep unsent attendance/marks drafts in local state until save succeeds (verify per screen), or persist drafts in AsyncStorage.
- **Effort:** M **Dependencies:** none.

MOB-010 — Client-side role selection (informational; backend enforces)

- **Category:** Mobile / Authorization **Severity:** P4 **Confidence:** High **Status:** Confirmed (no defect found)
- **Evidence:** `AppNavigator.js:282-285` chooses Teacher or Parent tabs from `state.auth.userRole`, rehydrated from plain AsyncStorage `user_role` (`App.js:46-56`). Login trusts the response role (`authApi.js:34`, staff uses backend `roleKey`). Backend: `app/api/auth/teacher/login/route.js` signs the role into the JWT after credential validation with rate limiting; `lib/auth/session.js:30-36` reads the Bearer token; the register-token route uses `getCurrentUserInfo()` from the token (`app/api/notifications/register-token/route.js`).
- **Impact:** On a rooted device a user could edit `user_role` to see another role's screens, but every API call is authorised from the JWT. Per-endpoint authorization for all roles used by the app (`/exams`, `/attendance`, `/fees`, `/assessments`, `/students` and so on) was NOT verified here (backend auditors).
- **Recommended fix:** None required; keep authorization server-side. `user_data` in plain AsyncStorage is display-only (JWT is in SecureStore on native, `secureAuthStorage.js`; web fallback is AsyncStorage by design).
- **Effort:** n/a **Dependencies:** none.

MOB-011 — Generated manifest requests extra permissions

- **Category:** Mobile / Privacy **Severity:** P4 **Confidence:** Low **Status:** Suspected
- **Evidence:** `android/app/src/main/AndroidManifest.xml:4-5` lists `RECORD_AUDIO` and `SYSTEM_ALERT_WINDOW`; `app.json` declares neither, and `/android` is git-ignored (regenerated by prebuild/EAS), so the release manifest was not verified. `SYSTEM_ALERT_WINDOW` usually comes from dev-client debug builds.
- **Impact:** If present in a release build, over-broad permissions (Play review, user trust).
- **Recommended fix:** Add `android.blockedPermissions` in `app.json` for those two and inspect the release AAB manifest.
- **Effort:** S **Dependencies:** none.

MOB-012 — Very large screen files

- **Category:** Mobile / Maintainability **Severity:** P3 **Confidence:** Medium **Status:** Confirmed
- **Evidence:** `ParentCalendarScreen.js` 1,061 lines, `EnterMarksScreen.js` 1,039, `DailyAttendanceScreen.js` 1,008, `ParentAttendanceScreen.js` 895, `TeacherDashboardScreen.js` 822.
- **Impact:** Hard to test/review; style blocks mixed with logic.
- **Recommended fix:** Split when touched; no rewrite.
- **Effort:** L **Dependencies:** MOB-007.

Positive observations

- JWT stored via `expo-secure-store` on native with a documented web fallback (`secureAuthStorage.js`).
- Retry policy is idempotent-only: GET on transient errors, never on mutations (`baseApi.js:25-37`); 30 s timeout set.
- Dev-only request/response logging gated by `__DEV__` (`baseApi.js:44-58`); push logs also `__DEV__`-gated.
- Sentry enabled only with env flag + DSN, disabled in `__DEV__`; React error boundary with fallback (`App.js`). No `sendDefaultPii`.
- Offline banner via NetInfo; push permissions requested only after login; receipt HTML for `expo-print` escapes values (`feesReceipt.js`, `esc(...)` on interpolated fields).
- Razorpay: amount and order come from the server; app only holds the public key id.

- Service-account key and release keystore are git-ignored; `app.json` has a unique `android.package`.
- Server enforces auth from JWT; teacher login is rate limited.

Limitations / not assessed

- No device/emulator run, no build, no `npm test` / `npm audit` for the mobile project (not permitted/needed for audit); dependency vulnerabilities not assessed.
- Contents of `eas.json`, `google-services.json`, adminsdk JSON, keystore, `sentry.properties`, `.env.local` deliberately not read; `eas.json` secrets/profile separation (API URL per environment) therefore not verified beyond key names.
- Deep-link configuration: only the `schoolapp360` scheme in `app.json`; `NavigationContainer` has no `linking` config and notification taps just navigate to the Notifications screen (no deep-link exposure found). iOS/Play store build config, OTA updates (`expo-updates` is not installed) and certificate pinning not assessed.
- Per-endpoint backend authorization for mobile routes not verified (see MOB-010); screens other than auth, fees and push were skimmed by grep only.

11 — Observability, operations and privacy audit

Auditor: A6 (prefixes OBS, PRV). Scorecard areas: **Observability**, **Privacy and operational readiness**.

Scope

ADMIN (`adminpanel`): logging, error tracking, cron/job health, notification delivery, payment reconciliation, audit trails, health checks, backup/incident documentation; personal-data inventory, storage, access control, retention/erasure, consent/notice, processors. MOBILE inspected only for Sentry/consent presence. Legal conclusions (DPDP Act 2023, children's data, cross-border) are **not** made here: they are flagged for legal verification.

Method

- Listed all `lib/` and `app/api` route groups; grepped every `console.*` in `lib/`, `app/api`, `app/actions` (30 sites) and read each context.
- Read `lib/expoPush.js`, `lib/pusher.js`, `lib/adminNotifications.js`, `lib/email.js`, `lib/storage.js`, `lib/auth/encryption.js`, both cron routes, `lib/fees.js` (cron + Razorpay verify), `vercel.json`, `next.config.mjs`, `middleware.js`.
- Read `prisma/schema.prisma` Student (357-448), Teacher (~600-660), ParentAccount (271-286), Payment (1427), ExamAuditLog (1297) and RBAC grants (`prisma/rbacPermissions.js`).
- Traced student/teacher read routes (`app/api/students/route.js`, `paged`, `[id]`, `print-details`, `app/api/teachers/*`) and both document-upload routes.
- Verified vs only read: **all code-based; nothing executed**. No HTTP requests were made. Git-tracking checks (`git ls-files`, `git check-ignore`) were run.
- Sampling: 26 route files opened for this report plus the 16 for report 14.

Summary table

ID	Sev	Conf	Title	Component
PRV-001	P1	High	Parent and Accountant roles can read any student's full record (Aadhaar, guardians, address)	`GET /api/students/[id]`
PRV-002	P1	Medium	Aadhaar and identity documents stored under permanent unsigned public URLs	R2 storage / document upload
OBS-003	P1	High	Daily fee-generation cron only processes one fallback school, not every school	`api/cron/generate-fees`, `lib/fees.js`
OBS-007	P1	High	No Razorpay webhook or reconciliation; paid-but-unrecorded payments possible	Payments
PRV-003	P2	High	Aadhaar, PAN and bank details stored and served in plaintext	Student/Teacher models
PRV-004	P2	High	No erasure/retention path for student/teacher personal data and documents	Delete flows
PRV-005	P2	High	No privacy notice, terms or consent capture anywhere (web and mobile)	Whole product
OBS-001	P2	High	No server-side error tracking, request IDs or structured logs on the web app	Observability
OBS-004	P2	High	Cron endpoints fail open if `CRON_SECRET` is unset	`api/cron/*`
OBS-005	P2	High	Cron/job failures are swallowed: HTTP 200 and no alert	`generateDueStudentFees`

ID	Sev	Conf	Title	Component
OBS-006	P2	High	Push / realtime / in-app notification failures are console-only, no retry or record	Notifications
OBS-008	P2	High	Most sensitive actions have no audit trail	Fees, roles, deletes, settings
OBS-010	P2	Medium	No backup/restore or incident-response documentation	Operations
PRV-006	P3	Medium	Third-party processors and hosting region need a documented basis	Processors
PRV-007	P3	Medium	Unredacted error objects logged; token logged in push errors	Logging
OBS-002	P3	High	No `error.jsx` / `global-error.jsx` boundaries	App shell
OBS-009	P3	High	No health/uptime endpoint or DB metrics	Operations
OBS-011	P3	High	Dev-email fallback prints recipient and full message body to logs	`lib/email.js`
OBS-012	P3	High	`dev.log` is tracked in git	Repo hygiene
OBS-013	P3	Medium	Login abuse limiter is per-instance memory with no alerting or failed-login record	Auth abuse
PRV-008	P4	Medium	Realtime channels are public with predictable names (ID-only payloads)	Pusher

Findings

PRV-001 — Parent and Accountant roles can read any student's full record

- **Category:** Security / Privacy
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `GET /api/students/[id]`, `lib/students.js` `getStudentById` / `decorateStudent`
- **Evidence:** `app/api/students/[id]/route.js:7-19` — gate is `requirePermission('students.view')` followed by a block that rejects only `user.roleKey === 'Teacher'`; the comment (lines 10-16) states the route is meant to be "admin-tier only". `prisma/rbacPermissions.js` grants `students.view` to Parent (line ~205) and Accountant (line 195) as well as Teacher/Admin. `lib/students.js:310-312` returns the full decorated record by `{id, schoolId}`, i.e. including `aadhaarNumber`, guardian/secondary guardian (phones, emails, Aadhaar), address, documents URLs (schema `Student` 372-405).
- **Impact:** A signed-in parent can fetch any other student in the same school (ids are cuids, but they are exposed in rosters, notices, links) and read minors' Aadhaar numbers, guardian contacts and addresses. Accountant (no teaching/admin need) gets the same. Same-school only; no cross-tenant leak.
- **Reproduction / verification:** code-based; not executed. Safe check on a test school: sign in as a Parent, GET `/api/students/<another student id>` and observe 200 with full fields.
- **Root cause:** Deny-list (`Teacher`) instead of an allow-list on a route whose permission is shared by low-privilege roles.
- **Recommended fix:** In the route, allow only roles that legitimately need the full record: `if (!['Admin','Principal','SuperAdmin'].includes(user.roleKey)) return 403`; Parent already has dedicated parent routes. Consider returning a reduced projection for Accountant.
- **Validation:** Parent/Accountant/Teacher get 403; Admin/Principal 200.
- **Effort:** S **Dependencies:** authorization auditor (shared RBAC grants); PRV-003

PRV-002 — Aadhaar and identity documents under permanent unsigned public URLs

- **Category:** Security / Privacy
- **Severity:** P1 **Confidence:** Medium **Status:** Suspected (bucket policy not inspected)
- **Affected component:** `lib/storage.js`, `app/api/students/document-upload-url/route.js`, `app/api/teachers/document-upload-url/route.js`
- **Evidence:** `lib/storage.js:20-27` — `buildPublicUrl` returns `${R2_PUBLIC_URL}/${key}` and its comment states the bucket must allow public reads; there is no signed-GET path in the file. Student document keys are `students/<schoolId>/documents/<Date.now()>-<sanitised original filename>` (`students/document-upload-url/route.js:26-28`); teacher equivalent `teachers/document-upload-url/route.js:22-24`. The URL is stored in the DB (`Student.aadhaarDocumentUrl` schema 383, Teacher 629) and returned to API callers (see PRV-001).
- **Impact:** Anyone holding or guessing a URL can download a child's or teacher's Aadhaar/birth-certificate scan with no authentication, no expiry and no access log. Keys are not high-entropy (millisecond timestamp plus a user-chosen filename such as the document type), and URLs leak through any log, referrer, shared screenshot or the PRV-001 route.
- **Reproduction / verification:** code-based; not executed. Confirm in the Cloudflare dashboard whether public access is enabled for the bucket/domain.
- **Root cause:** Reuse of the public-read pattern designed for notice PDFs and profile photos for identity documents.
- **Recommended fix:** Smallest change: keep photos public, but store identity documents under a separate private bucket (no public domain) with a random key (`crypto.randomUUID()`), and serve them via a permission-checked route that returns a short-lived presigned GET (`GetObjectCommand` + `getSignedUrl`, 60-300 s). Existing objects need migration.
- **Validation:** Unauthenticated GET of a document URL returns 403/404; signed link works only for authorised roles and expires.
- **Effort:** M **Dependencies:** PRV-001, PRV-004

OBS-003 — Fee-generation cron only processes one fallback school

- **Category:** Backend / Operations
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `app/api/cron/generate-fees/route.js`, `lib/fees.js:585-607`, `lib/auth/schoolContext.js:18-27`
- **Evidence:** `generateDueStudentFees()` calls `getFeeStructures()` (no school argument); that resolves school through `resolveSchoolId()`, which with no session (a cron has none — the function's own comment at `fees.js:579-583` says so) returns the process-wide `getActiveSchoolId()` singleton (`schoolContext.js:26`, default seed id at `lib/school.js:5,17`).
- **Impact:** On a multi-school deployment the daily job generates term fees for at most one school (whichever the instance last "stepped into", else the seed id); all others are silently skipped while the route returns `success: true`. Parents of other schools are not billed on time; no one is alerted.
- **Reproduction / verification:** code-based; not executed (cron routes must not be called). Read the call chain above; on a DB with two schools only one school's `StudentFee` rows are produced on a cron day.
- **Root cause:** Tenant resolved from ambient state instead of iterating schools explicitly in the job.
- **Recommended fix:** In the cron, `const schools = await prisma.school.findMany({select:{id:true}})` and call a school-parameterised generator for each, collecting per-school success/failure in the response.
- **Validation:** With two seeded schools having due structures, both get fees in one run (on a non-production DB); response lists per-school counts.
- **Effort:** M **Dependencies:** OBS-005; prior audit P13

OBS-007 — No Razorpay webhook or reconciliation tooling

- **Category:** Backend / Financial integrity
- **Severity:** P1 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `app/api/payments/razorpay/*`, `lib/fees.js` (~1190-1275)
- **Evidence:** only `create-order` and `verify` routes exist under `app/api/payments/` (directory listing); `lib/fees.js:1191,1257` comments say a webhook is a future addition ("once built"). Payment rows start `PENDING` (`schema.prisma:1437`) and become `SUCCESS` only when the browser/app calls `verify` (`fees.js:1236-1248`). No job re-queries Razorpay for stale `PENDING` rows, and no admin screen lists them. Also `create-order/route.js:13` and `verify/route.js:12` skip the ownership check when no actor is signed in.
- **Impact:** If a parent pays and the app/tab closes, the network drops, or the callback fails, money is captured at Razorpay but the fee stays unpaid and the Payment stays `PENDING` indefinitely; staff must reconcile manually against the Razorpay dashboard. Refunds do not exist either (prior audit P28). The null-actor path lets an unauthenticated caller create orders/PENDING rows against any fee id (signature check still protects `verify`).
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Client-driven confirmation only.
- **Recommended fix:** Add `POST /api/payments/razorpay/webhook` (verify `X-Razorpay-Signature` with the webhook secret over the raw body; call the existing idempotent confirm path which already tolerates duplicates, `fees.js:1256-1262`), plus a daily/admin "stale PENDING payments" check that fetches the order status. Require a signed-in actor on `create-order` / `verify`.
- **Validation:** Replay a webhook twice (test mode): one SUCCESS row, fee marked once; stale PENDING list is empty after reconciliation.
- **Effort:** M **Dependencies:** OBS-005

PRV-003 — Aadhaar, PAN and bank details stored and served in plaintext

- **Category:** Security / Privacy
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `Student`, `Teacher` models; `lib/teachers.js` `decorateTeacher`
- **Evidence:** `schema.prisma:376` (`Student.aadhaarNumber String`), guardian Aadhaar inside `guardian` / `secondaryGuardian` Json (`lib/students.js:319-333`), `Teacher.aadhaarNumber` (624), `panNumber` (634), `bankDetails Json` (637). `lib/auth/encryption.js` (AES-256-GCM) exists but is used only for the Razorpay key secret (`lib/razorpay.js:4`, `lib/schools.js:2`). `lib/teachers.js:103,110,119` return Aadhaar, PAN and bank details; `GET /api/teachers/paged` (`paged/route.js:18-27`) returns that full decorated shape for every teacher on every page to any holder of `teachers.view` (Admin/Principal).
- **Impact:** A database leak, backup exposure or an over-broad read exposes government IDs and bank accounts directly; list views carry far more than a table needs.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Identifiers treated as ordinary profile fields.
- **Recommended fix:** (1) Return a list projection without Aadhaar/PAN/bank from `paged`; fetch sensitive fields only on a detail view. (2) Reuse `encryptSecret` / `decryptSecret` for `aadhaarNumber`, `panNumber` and bank account number, and display masked (last 4) by default. Requires a one-off data migration; search by these fields is not currently offered.
- **Validation:** DB rows hold `iv:tag:cipher` strings; list API has no such fields.
- **Effort:** M **Dependencies:** PRV-001, PRV-002

PRV-004 — No erasure/retention path for personal data and documents

- **Category:** Privacy / Data lifecycle
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based; legal need requires verification)
- **Affected component:** `lib/students.js:673-698`, teachers routes, storage
- **Evidence:** `deleteStudent` removes the row and only the profile photo object (`students.js:693-696`); Aadhaar, guardian and other document URLs in `documents` / `aadhaarDocumentUrl` /guardian Json are never deleted from

R2. A student with any fee/payment row cannot be deleted at all (Restrict, `:686-690`); the suggested alternative is status change, which keeps all personal data. No teacher delete route exists (`app/api/teachers/[id]` has `assignments`, `reset-password`, `status`, `route.js` with PUT only). Only support tickets have a retention job (`lib/support.js:234-254`, 90 days). No data-export (access) function and no anonymisation function were found.

- **Impact:** Personal data of children and staff persists indefinitely, and orphaned identity documents remain publicly reachable (PRV-002) even after a "delete". If the operator must honour deletion/withdrawal requests, there is no supported way to do it.
- **Reproduction / verification:** code-based; not executed.
- **Root cause:** Lifecycle was not designed; deletion handles only the row and photo.
- **Recommended fix:** On delete/transfer-out purge document objects (`documents.*Url`, `aadhaarDocumentUrl`, guardian document URLs) via `deleteObject`; add an `anonymizeStudent` that blanks identifiers but keeps fee/payment rows; define a retention schedule with the school (fee records often have statutory retention — verify).
- **Validation:** After purge, R2 keys are gone and API returns blanks.
- **Effort:** M **Dependencies:** PRV-002, PRV-005

PRV-005 — No privacy notice, terms or consent capture

- **Category:** Privacy / Compliance
- **Severity:** P2 **Confidence:** High **Status:** Confirmed that none exists in code; compliance conclusion requires legal verification
- **Affected component:** web `app/` and components; mobile `src/`
- **Evidence:** case-insensitive grep for privacy/consent/terms across `app` and `components` finds only fee-term wording; `schoolapp360/src` and `app.json` contain no privacy/consent references. Login pages, parent account creation (`lib/parentAccounts.js`) and student admission forms carry no notice or consent record; there is no consent field in `schema.prisma`.
- **Impact:** The product processes data of minors (DOB, Aadhaar, caste, religion, blood group, guardian contacts). Under the Digital Personal Data Protection Act 2023, personal data of persons under 18 generally requires verifiable parental/guardian consent and a notice; tracking/behavioural monitoring of children is restricted. Whether the school (data fiduciary) or SchoolApp360 (processor) must collect that here, and in what form, is a legal question; code alone shows no mechanism. Not a compliance conclusion. Mobile stores crash data via Sentry (see PRV-006); no ad/behavioural analytics SDKs were found in either `package.json`.
- **Reproduction / verification:** code-based.
- **Root cause:** Feature scope has not covered notices/consent.
- **Recommended fix:** After legal advice: publish privacy notice/terms pages, link from login screens and app stores, add a recorded acceptance (timestamp, version) at parent first-login and a school-level data-processing agreement. Data minimisation: make caste/religion/Aadhaar optional per school.
- **Validation:** Acceptance rows exist; notice reachable unauthenticated.
- **Effort:** M **Dependencies:** PRV-004

OBS-001 — No server-side error tracking, request IDs or structured logs (web)

- **Category:** DevOps / Observability
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** web app (`package.json`, `next.config.mjs`, `lib/*`)
- **Evidence:** no Sentry/APM/logging library in `package.json`; no `instrumentation.js`; `next.config.mjs` is empty; grep for `requestId|x-request-id|correlation` in `lib`, `app`, `middleware.js` -> none. All 30 log sites are free-text `console.error('<name> failed', err)` (e.g. `lib/fees.js:567,599,1061`, `lib/pusher.js:35`, cron routes `generate-fees/route.js:21`). The mobile app does have Sentry (`schoolapp360/src/core/sentry.js:10-22`, enabled only by env flags).

- **Impact:** Production errors on API routes and server components (payments, fee generation, notifications) are visible only in Vercel's raw function logs (retention limited by plan); no grouping, alerting, release tagging or per-request tracing; the mobile crash view cannot be correlated to a server error.
- **Reproduction / verification:** code-based; Vercel-side configuration not inspected.
- **Root cause:** Observability not wired.
- **Recommended fix:** `@sentry/nextjs` with `instrumentation.js` + `sendDefaultPii:false` and a `beforeSend` scrubber (see PRV-007); add a tiny `lib/log.js` that emits one JSON line `{level, event, schoolId, route, err: err.message}` and reuse it in the 30 sites; propagate a request id from `middleware.js` (`crypto.randomUUID()` into a response header).
- **Validation:** Throw in a test route -> event appears with route and school tags and no PII.
- **Effort:** M **Dependencies:** PRV-007

OBS-004 — Cron endpoints fail open if `CRON_SECRET` is unset

- **Category:** Security / Operations
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `app/api/cron/generate-fees/route.js:12-15`, `cleanup-support-tickets/route.js:8-11`
- **Evidence:** the check is `if (process.env.CRON_SECRET && authHeader !== ...)`, so with the variable absent the guard is skipped and anyone can GET the route. Whether production sets it was not verifiable.
- **Impact:** If the variable is missing, any internet user can trigger fee generation or delete 90-day-old support tickets (idempotent-ish, but still unintended mass jobs) and obtain per-structure result JSON.
- **Reproduction / verification:** code-based only (cron routes must not be called in this audit).
- **Root cause:** Optional-secret pattern.
- **Recommended fix:** `if (!process.env.CRON_SECRET || authHeader !== \`Bearer ${process.env.CRON_SECRET}\`) return 401`.
- **Validation:** Request without header returns 401 even when env is unset.
- **Effort:** S **Dependencies:** none

OBS-005 — Cron/job failures are swallowed

- **Category:** Backend / Operations
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `lib/fees.js:597-601`, both cron routes
- **Evidence:** per-structure errors are caught and logged with `console.error` then dropped; the route still returns `{success:true, termsProcessed}` (`generate-fees/route.js:19`). Failed terms are not recorded in results, so the Vercel cron dashboard shows success. Idempotency is reasonable (`@@unique([studentId, academicSession, term])`, schema 556) so a re-run is safe.
- **Impact:** A broken structure silently leaves a class unbilled; discovery is by parent complaint.
- **Reproduction / verification:** code-based.
- **Root cause:** No failure accounting or alert channel.
- **Recommended fix:** push `{structureId, term, error: err.message}` into a `failures` array, return HTTP 500 when non-empty, and notify via an alert (Sentry event or admin notification to SuperAdmin).
- **Validation:** Force one failing structure on a test DB: response 500 and alert fires.
- **Effort:** S **Dependencies:** OBS-001, OBS-003

OBS-006 — Notification delivery failures are console-only

- **Category:** Backend / Operations
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** `lib/expoPush.js`, `lib/pusher.js`, `lib/adminNotifications.js`, `lib/examNotifications.js`

- **Evidence:** `expoPush.js:41-61` handles `DeviceNotRegistered` by deleting the token (good) but logs every other ticket error and batch failure to console and returns; no receipts check (Expo's receipt API is never queried), no retry. `pusher.js:31-37` and `adminNotifications.js:38-42` swallow errors after `console.error`. In-app notification inserts are separate DB rows so they still exist when push fails.
- **Impact:** Parents can miss fee/notice/homework pushes with no record visible to staff; a misconfigured Expo/FCM credential would fail every push silently until someone reads logs.
- **Reproduction / verification:** code-based.
- **Root cause:** Best-effort design without a failure signal.
- **Recommended fix:** minimum: count failures per run and report them through the Sentry/log helper from OBS-001 with an alert threshold; optional later: query Expo receipts. No queue infrastructure needed.
- **Validation:** Bad Expo credential produces an alert within one send.
- **Effort:** S-M **Dependencies:** OBS-001

OBS-008 — Most sensitive actions have no audit trail

- **Category:** Security / Operations
- **Severity:** P2 **Confidence:** High **Status:** Confirmed (code-based)
- **Affected component:** whole API
- **Evidence:** audit-style models: `ExamAuditLog` (`schema.prisma:1297`, written via `logExamAudit` from exam/mark/result/schedule libs only), `StudentPromotionLog` (1041), `MonthlyReportActivity` (1565). `ExamAuditLog` is `onDelete: Cascade` from `Exam` (line ~1302), so deleting an exam deletes its audit history. `Payment` has no recorder/actor column (`schema.prisma:1427-1443`), so a cash/UPI payment recorded by staff has no "who". No audit rows for: fee discount apply/remove, fee structure changes, payment collection, user/role creation and role change (`app/api/users/[id]/role/route.js`), password resets (admin, teacher, parent, student portal), student/teacher create-update-delete, bulk import, academic session activation, school settings and branding changes, SuperAdmin "step into school" (`schools/[id]/manage/route.js`), login success/failure, data views of Aadhaar fields.
- **Impact:** Cannot answer "who changed this fee / role / record" after an incident or dispute; no evidence trail for fraud investigation or for responding to a privacy request.
- **Reproduction / verification:** code-based (grep for audit/log model writes outside the three above).
- **Root cause:** Audit was built only for the exam module.
- **Recommended fix:** one generic `AuditLog` model (`schoolId, actorId, actorRole, action, entityType, entityId, meta Json, createdAt`) and a 10-line `logAudit()` helper (same pattern as `lib/examAuditLog.js`), called from the high-value mutation points first: payments/discounts, role changes, password resets, student/teacher delete, settings. Add `recordedBy` to `Payment`. Remove cascade on ExamAuditLog or copy before delete.
- **Validation:** each listed action creates one row with actor.
- **Effort:** M **Dependencies:** none

OBS-010 — No backup/restore or incident-response documentation

- **Category:** DevOps / Operations
- **Severity:** P2 **Confidence:** Medium **Status:** Not verified
- **Affected component:** operations
- **Evidence:** `docs/` contains only `API-Spec.md`; `README.md` and `docs/` grep for backup/restore/incident/runbook/disaster -> no matches. Database provider backup settings (Supabase) are outside the repo and were not accessible.
- **Impact:** No documented RPO/RTO, restore drill, breach-response contacts or notification procedure; the mobile app and the web app share one database, so a bad migration or data loss affects all schools. Breach-notification duties (if any) need legal confirmation.
- **Reproduction / verification:** not verified; needs access to Supabase/Vercel/R2 consoles.

- **Root cause:** Not yet written.
- **Recommended fix:** one-page runbook: backup tier and retention, last restore test date, who to call, steps to rotate JWT/R2/Razorpay secrets, template for school/parent notification. Do a restore drill into a scratch database.
- **Validation:** Restore drill completed and dated.
- **Effort:** S-M **Dependencies:** OBS-009

PRV-006 — Third-party processors and hosting region need a documented basis

- **Category:** Privacy
- **Severity:** P3 **Confidence:** Medium **Status:** Not verified
- **Affected component:** Razorpay, Pusher, Expo Push, SMTP, Cloudflare R2, Sentry (mobile), Vercel/Supabase hosting
- **Evidence:** data sent (code-read): Expo push — device token, title, body such as the payment amount, and `data:{type,studentId}` (`lib/fees.js:1006-1012`); notice/homework/exam pushes similar. Pusher — IDs and status only (`lib/support.js:146,180-182,212`), no names or message text. SMTP — teacher invite/reset link emails (`lib/teachers.js:45`, `lib/emailTemplates.js:19+`). Razorpay — fee amount/order data. R2 — photos, identity documents, notice PDFs, support attachments. Mobile Sentry — crash/perf traces (`sendDefaultPii` is not set). `vercel.json:2` pins region `syd1`; the prior audit noted a Supabase Sydney database.
- **Impact:** Indian students' data is processed on infrastructure outside India and by several processors; whether that is permitted and what contracts/notices are required is a legal question (DPDP cross-border rules depend on government notifications). Not concluded here.
- **Reproduction / verification:** hosting regions to be confirmed in provider dashboards.
- **Root cause:** No processor register.
- **Recommended fix:** maintain a short processor/region table in the privacy notice and DPA; confirm R2 jurisdiction and Supabase region; consider India-region hosting if legal advice requires.
- **Validation:** Table reviewed by counsel.
- **Effort:** S **Dependencies:** PRV-005

PRV-007 — Unredacted error objects logged; push token in error log

- **Category:** Privacy / Observability
- **Severity:** P3 **Confidence:** Medium **Status:** Suspected
- **Affected component:** 30 `console.error` sites
- **Evidence:** `lib/expoPush.js:53-55` logs the recipient Expo push token (a device identifier) on every ticket error; many sites pass the whole `err` object (`lib/fees.js:567,599,1061`, `lib/notices.js:523`, `lib/parentNotifications.js:37`). Prisma error objects can embed query arguments. I did not find a site that logs request bodies, passwords, OTPs or Aadhaar values directly. No redaction layer exists.
- **Impact:** Low today; becomes a PII/identifier leak into log storage if a failing query includes student fields.
- **Reproduction / verification:** code-based.
- **Root cause:** Free-form logging.
- **Recommended fix:** log `err.code` / `err.message` via the helper from OBS-001, drop the token from line 53-55 (log batch index only).
- **Validation:** grep shows no raw `err` objects in logs.
- **Effort:** S **Dependencies:** OBS-001

OBS-002 — No error boundaries

- **Category:** Frontend / Observability
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Affected component:** `app/`

- **Evidence:** no `error.jsx` or `global-error.jsx` anywhere under `app` (only `app/not-found.jsx`). Same as prior audit P34.
- **Impact:** Unhandled render errors show the framework default page and are not reported anywhere (no hook to Sentry).
- **Reproduction / verification:** code-based.
- **Root cause:** Never added.
- **Recommended fix:** add `app/global-error.jsx` and `app/error.jsx` that call the error reporter and show a friendly retry.
- **Validation:** throw in a page; fallback UI and event appear.
- **Effort:** S **Dependencies:** OBS-001

OBS-009 — No health endpoint or DB metrics

- **Category:** DevOps
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Affected component:** `app/api`
- **Evidence:** `app/api` has no health/status route (directory listing); no slow-query logging in `lib/db.js:6` (plain `new PrismaClient()`); no latency metrics.
- **Impact:** No external uptime probe target and no DB-liveness signal; latency issues visible only in Vercel's dashboard.
- **Reproduction / verification:** code-based.
- **Root cause:** Not built.
- **Recommended fix:** `app/api/health/route.js` doing `prisma.$queryRaw\`SELECT 1\`` returning 200/503 (no data, no auth); point an uptime monitor at it.
- **Validation:** Probe goes red when DB is unreachable.
- **Effort:** S **Dependencies:** none

OBS-011 — Dev-email fallback prints recipient and body to logs

- **Category:** Privacy / Operations
- **Severity:** P3 **Confidence:** High **Status:** Confirmed (conditional on SMTP_HOST unset)
- **Affected component:** `lib/email.js:23-34`
- **Evidence:** when `SMTP_HOST` is absent the transport is a stream transport, nothing is sent, and the function `console.log`s `To`, `Subject` and the full text body (password setup/reset links with tokens).
- **Impact:** If production lacks `SMTP_HOST`, invites silently do not deliver while valid password-set links and recipient addresses are written to Vercel logs.
- **Reproduction / verification:** code-based.
- **Root cause:** Dev convenience reachable in production.
- **Recommended fix:** guard with `NODE_ENV !== 'production'`; in production throw so the caller reports failure.
- **Validation:** Production build without SMTP errors instead of logging.
- **Effort:** S **Dependencies:** none

OBS-012 — `dev.log` tracked in git

- **Category:** DevOps
- **Severity:** P3 **Confidence:** High **Status:** Confirmed
- **Affected component:** repo root
- **Evidence:** `git ls-files` lists `dev.log` (46 KB, committed in the initial commit); `.gitignore` ignores `npm-debug.log*` etc. but not `dev.log`. A count-only grep found no email addresses, "password" or "token" strings. `.env` is correctly untracked and ignored.
- **Impact:** Low: runtime logs in history may later contain data; no secrets found.

- **Reproduction / verification:** `git ls-files dev.log`.
- **Root cause:** Missing ignore rule.
- **Recommended fix:** `git rm --cached dev.log`, add `*.log` to `.gitignore` (owner action; not done here).
- **Validation:** file no longer tracked.
- **Effort:** S **Dependencies:** none

OBS-013 — Login abuse limiter is per-instance memory with no alerting

- **Category:** Security / Observability
- **Severity:** P3 **Confidence:** Medium **Status:** Confirmed (design limit stated in code)
- **Affected component:** `lib/auth/loginRateLimit.js:1-45`
- **Evidence:** in-memory `Map`; the file header acknowledges weakness on multi-instance serverless and cold starts. No record of failed logins or lockouts is persisted, nothing alerts on credential-spraying.
- **Impact:** Distributed or cold-start-resetting attacks are not slowed reliably and nobody is told they are happening.
- **Reproduction / verification:** code-based.
- **Root cause:** No shared store; accepted trade-off.
- **Recommended fix:** log a structured event on every lockout (Sentry/helper); if the abuse is observed, back the counters with a small Postgres table keyed by account/IP.
- **Validation:** lockout produces an alert-able event.
- **Effort:** S-M **Dependencies:** OBS-001, OBS-008

PRV-008 — Realtime channels are public with predictable names

- **Category:** Privacy
- **Severity:** P4 **Confidence:** Medium **Status:** Suspected
- **Affected component:** `lib/pusher.js:23-26`, `lib/support.js`
- **Evidence:** channels are `support-school-<schoolId>`, `support-superadmin`, `support-ticket-<ticketId>` (non-`private-`/`presence-` names, so no server auth endpoint is involved); payloads contain only ids/status. Not confirmed whether the client subscribes with an auth endpoint.
- **Impact:** A subscriber who knows a school or ticket id learns that activity occurred (no content). Low.
- **Reproduction / verification:** code-based.
- **Root cause:** Public channels for convenience.
- **Recommended fix:** keep payloads ID-only (as now) or move to `private-` channels with an auth route if content is ever added.
- **Validation:** n/a
- **Effort:** S **Dependencies:** none

Personal-data inventory (from `prisma/schema.prisma`)

Subject	Fields (sensitivity)	Storage	Who can read (code-based)
Student (minor)	name, DOB, gender, blood group, nationality, religion, caste, Aadhaar number, Aadhaar document URL, photo, WhatsApp, address, guardian x3 (phone, email, occupation, Aadhaar), emergency contacts, previous school, fee/discount details, birth certificate/SLC URLs	Postgres plaintext, documents on public R2	Admin/Principal (full); **Parent, Accountant via PRV-001**; Teacher gets 9 roster fields only (`students/route.js:16-26`, `paged/route.js:41-54`), plus DOB/guardian phone/address for own class via `print-details` (`print-details/route.js:29-62`, Class Teacher only)

Subject	Fields (sensitivity)	Storage	Who can read (code-based)
Teacher	DOB, Aadhaar, PAN, bank details, parents' details, qualifications, address, login email	plaintext Json/columns; documents on public R2	Admin/Principal via `teachers/paged` (full shape)
Parent account	email, name, password hash, photo, child links	Postgres, bcrypt hash	own account; staff
Payments	amount, method, Razorpay ids	Postgres	staff with fees permissions; no actor column
Push tokens	Expo token per owner	Postgres (`pushTokens`)	server only
School Razorpay secret	AES-256-GCM encrypted	Postgres	server decrypt only

Data minimisation notes: Teacher list returns everything; student list for admins also returns the full decorated record (`app/api/students/route.js:34-36`, `getAllStudents`) including Aadhaar and guardian Aadhaar. No bulk export routes for student data exist (only blank bulk-import templates), which limits export risk; fee summary exports at `app/api/fees/students/summary/route.js` were not reviewed.

Positive observations (verified)

- Teacher roster routes strip to 9 fields at the query level (`students/route.js:16-26,48`) and `[id]` blocks Teacher (`students/[id]/route.js:17`).
- Passwords are bcrypt-hashed; temp-password and teacher-invite tokens are random (`lib/teachers.js:25` 32 random bytes); `passwordSetToken` unique with expiry columns; session cookie is httpOnly, secure in production, 7-day expiry (`lib/auth/session.js:10-20`).
- Razorpay key secrets encrypted at rest (`lib/auth/encryption.js`); Razorpay confirmation is idempotent on payment id (`lib/fees.js:1256-1262`).
- Support-ticket retention job with R2 attachment cleanup exists (`lib/support.js:234-254`).
- Realtime (Pusher) payloads carry ids only; push bodies contain no names (fee push uses amount only).
- `.env` is untracked and git-ignored; no analytics/advertising SDK is present in either `package.json` (no behavioural tracking observed).
- Exam module has an audit log with actor, role, action and meta (`schema.prisma:1297-1311`); cron routes at least check a bearer secret when configured; fees generation is unique-constrained so re-runs are safe.

Limitations / not assessed

- Production/Vercel/Supabase/Cloudflare/Razorpay dashboard settings (bucket public access, backups, log retention, alerts, env vars, hosting regions): not accessible; marked Not verified.
- Legal compliance (DPDP Act 2023, children's data, cross-border transfer, breach notification, retention periods): cannot be concluded from code; items framed for legal review.
- No HTTP, DB or cron execution was performed. Fee summary/CSV export route and some parent routes were only sampled.
- Mobile app reviewed only for Sentry and consent text; its local data storage and permissions are outside this report.
- Verification fragment note: while checking whether `.env` carries the "demo" label I displayed a prefix of one secret line in my tool output; no value is reproduced in this report, but the JWT secret should be rotated as a precaution regardless (also prior audit P15).